

HARI VIGNESH RAO 23BD1A052Q

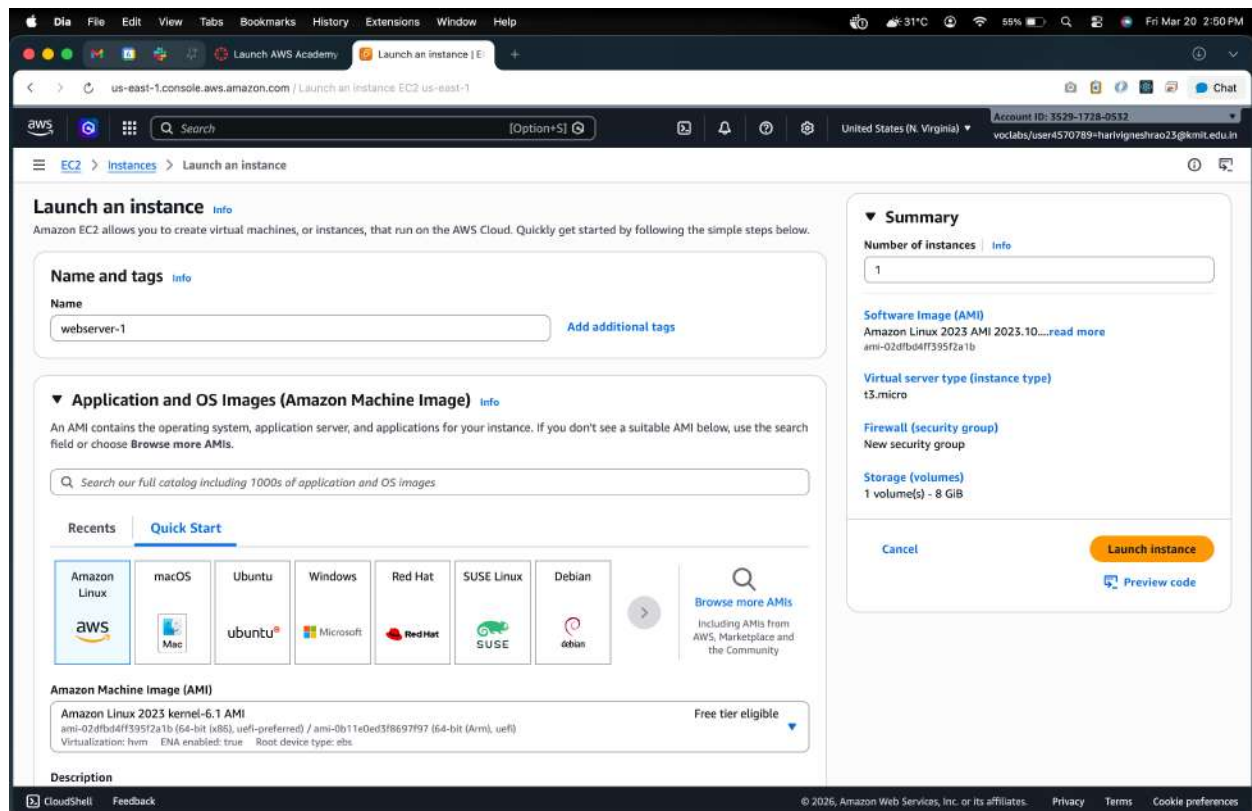
WEEK-9- Implementation of Elastic Load Balancer with Auto Scaling using AWS EC2 Instances.

Elastic Load Balancer (ALB)

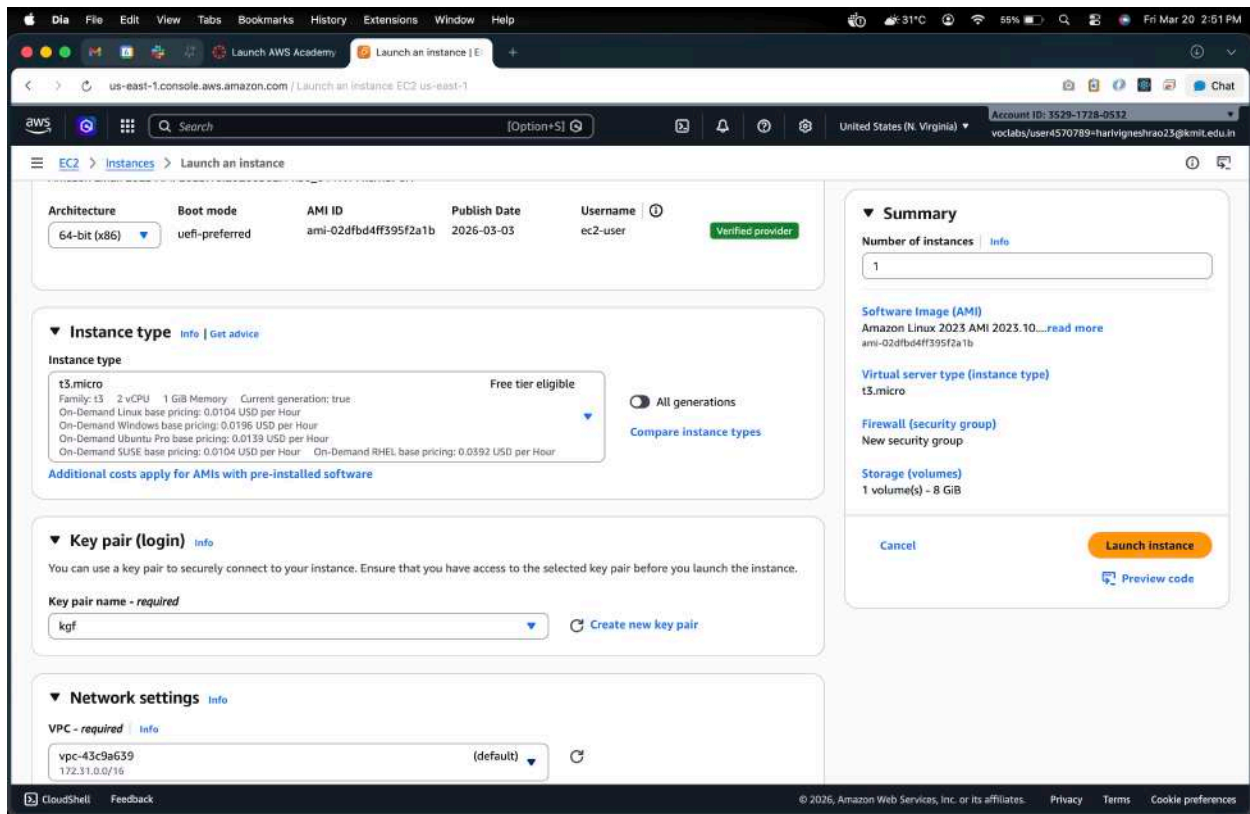
Step 1: Launch Two EC2 Instances

1.1 Launch EC2 Instance #1

1. Go to **AWS EC2 Console** → **Launch Instance**.
2. **Name:** webserver-1
3. Choose an **AMI** (Amazon Linux 2).



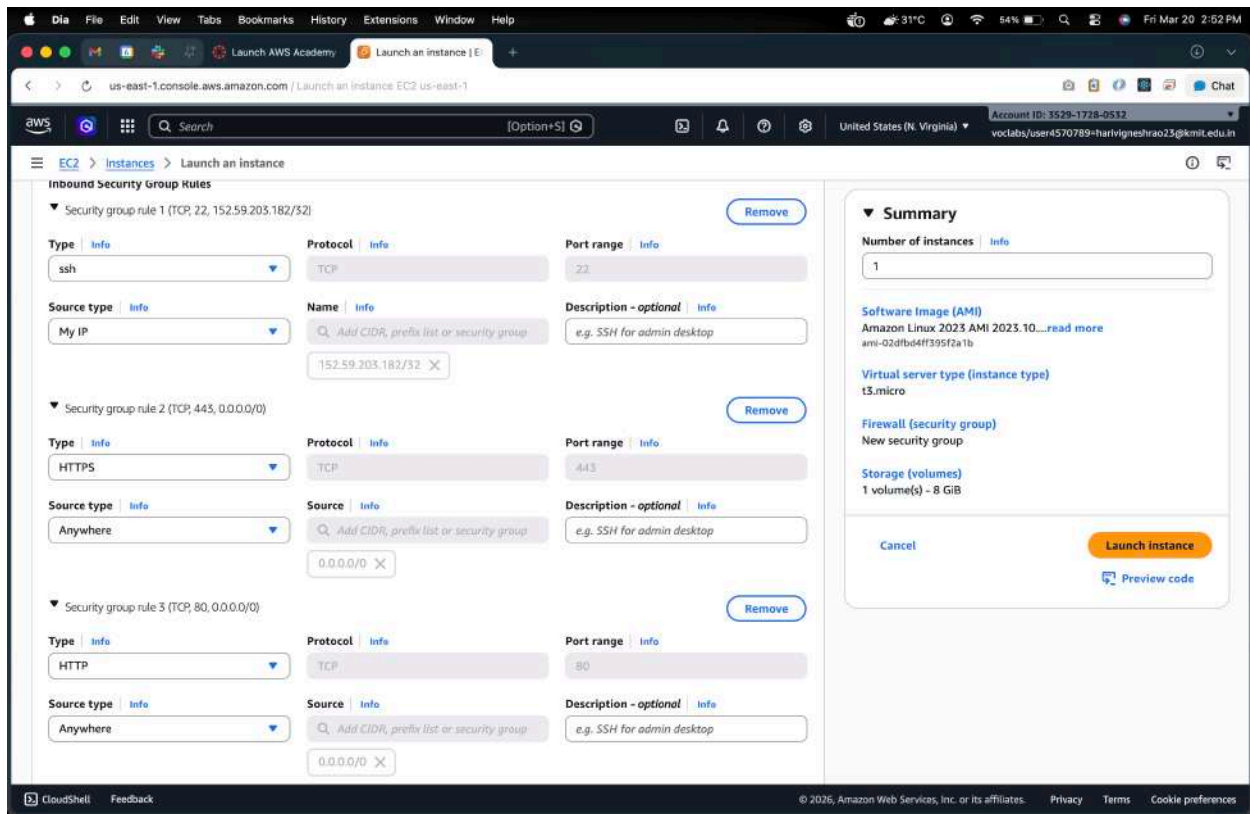
4. Select **Instance Type** (e.g., t2.micro).
5. Add **Storage** (default is fine).



6. Configure **Security Group**:

- Allow **SSH (22)** from your IP
- Allow **HTTP (80)** from anywhere (0.0.0.0/0)

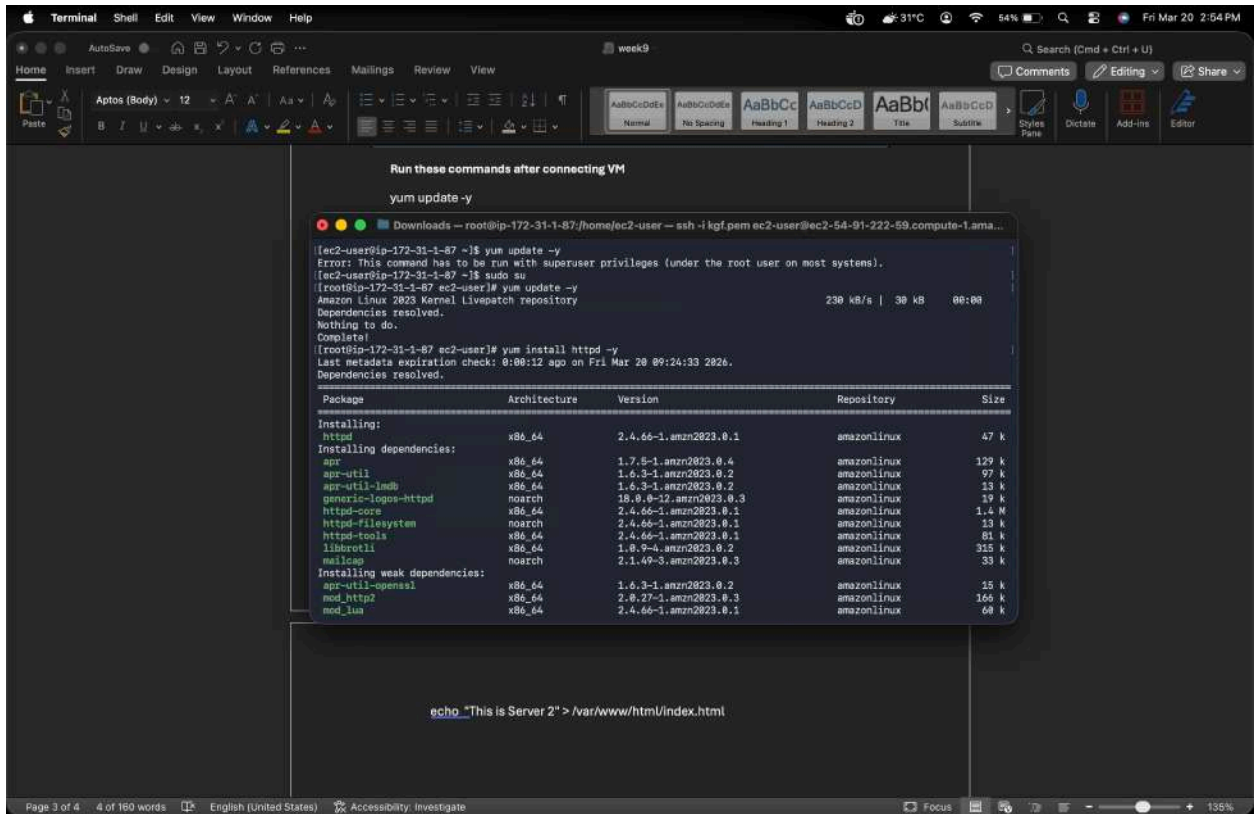
7. **Launch** → Select/Create Key Pair → Launch.



Run these commands after connecting VM

```
yum update -y
```

```
yum install httpd -y
```



```
Run these commands after connecting VM
yum update -y

[ec2-user@ip-172-31-1-87 ~]$ yum update -y
Error: This command has to be run with superuser privileges (under the root user on most systems).
[ec2-user@ip-172-31-1-87 ~]$ sudo su
[root@ip-172-31-1-87 ec2-user]# yum update -y
Amazon Linux 2023 Kernel Livepatch repository
Dependencies resolved.
Nothing to do.
Complete!
[root@ip-172-31-1-87 ec2-user]# yum install httpd -y
Last metadata expiration check: 0:00:12 ago on Fri Mar 20 09:24:33 2026.
Dependencies resolved.

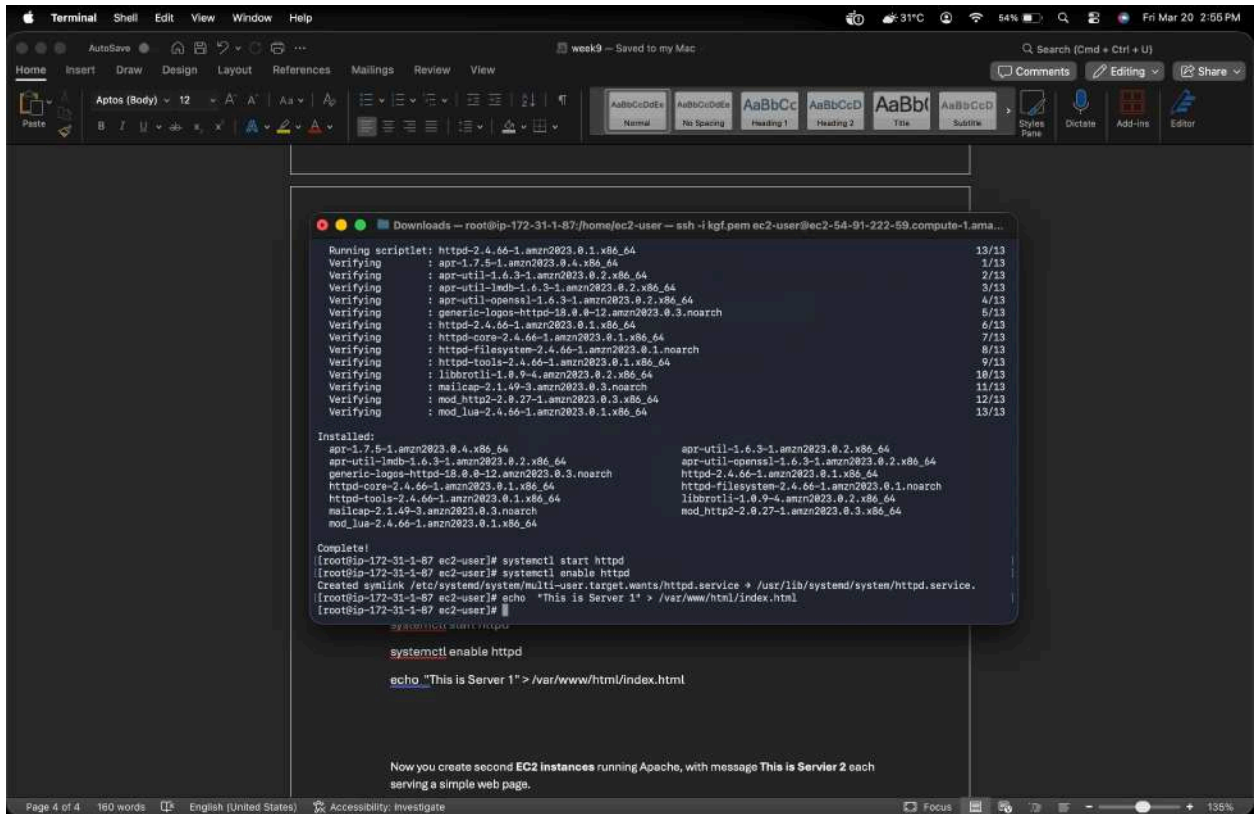
Package Architecture Version Repository Size
-----
Installing:
httpd x86_64 2.4.66-1.amzn2023.0.1 amazonlinux 47 k
Installing dependencies:
apr x86_64 1.7.5-1.amzn2023.0.4 amazonlinux 129 k
apr-util x86_64 1.6.3-1.amzn2023.0.2 amazonlinux 97 k
apr-util-ldap x86_64 1.6.3-1.amzn2023.0.2 amazonlinux 23 k
generic-logos-httpd noarch 15.0.0-12.amzn2023.0.3 amazonlinux 19 k
httpd-core x86_64 2.4.66-1.amzn2023.0.1 amazonlinux 1.4 M
httpd-filesystem noarch 2.4.66-1.amzn2023.0.1 amazonlinux 13 k
httpd-tools x86_64 2.4.66-1.amzn2023.0.1 amazonlinux 61 k
libtool x86_64 1.8.9-4.amzn2023.0.2 amazonlinux 355 k
mailcap noarch 2.1.49-3.amzn2023.0.3 amazonlinux 33 k
Installing weak dependencies:
apr-util-openssl x86_64 1.6.3-1.amzn2023.0.2 amazonlinux 25 k
mod_http2 x86_64 2.0.27-1.amzn2023.0.3 amazonlinux 166 k
mod_lua x86_64 2.4.66-1.amzn2023.0.1 amazonlinux 60 k

echo "This is Server 2" > /var/www/html/index.html
```

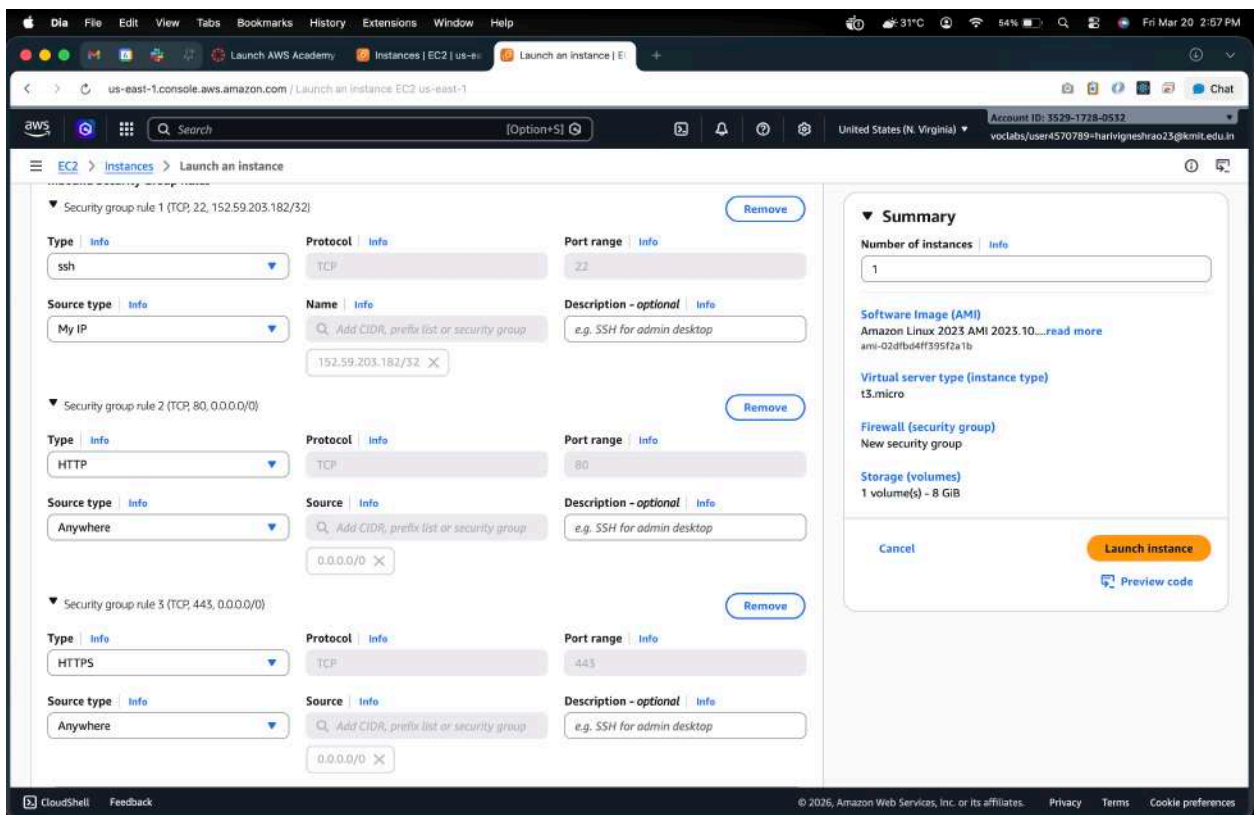
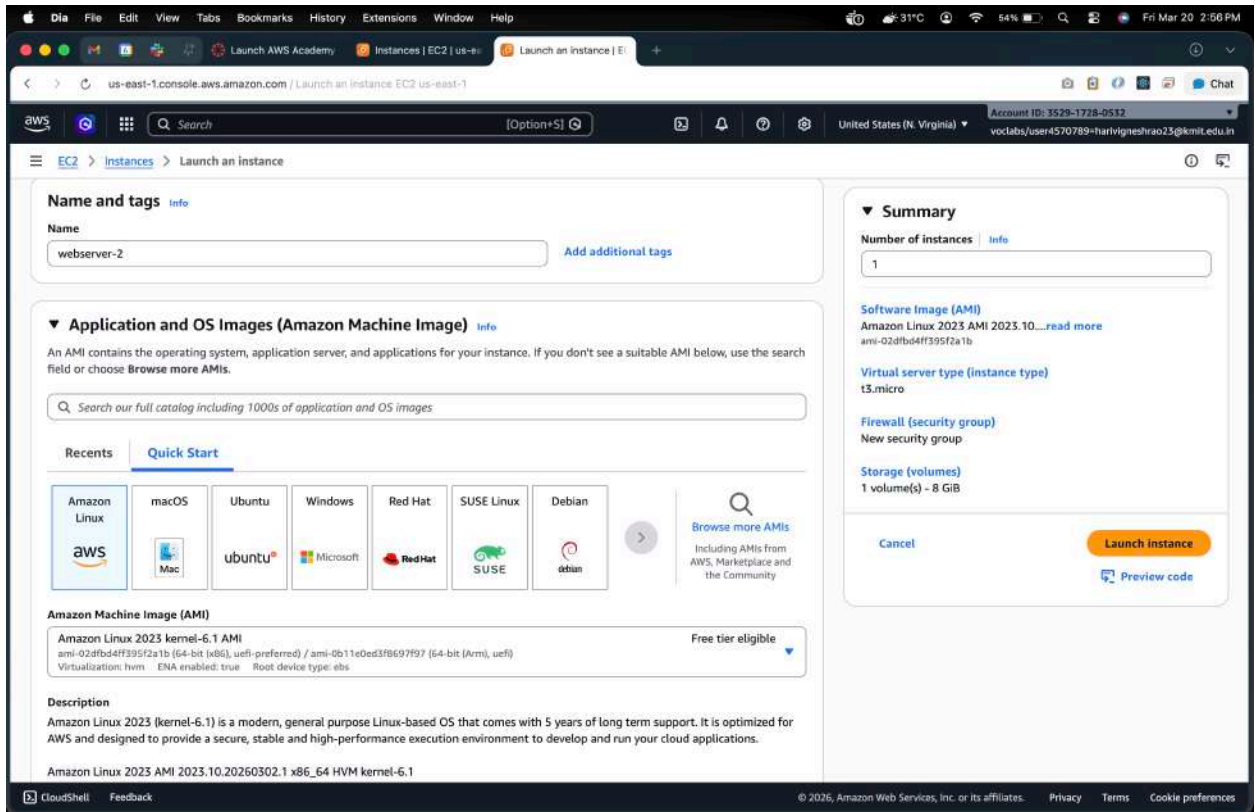
systemctl start httpd

systemctl enable httpd

echo "This is Server 1" > /var/www/html/index.html

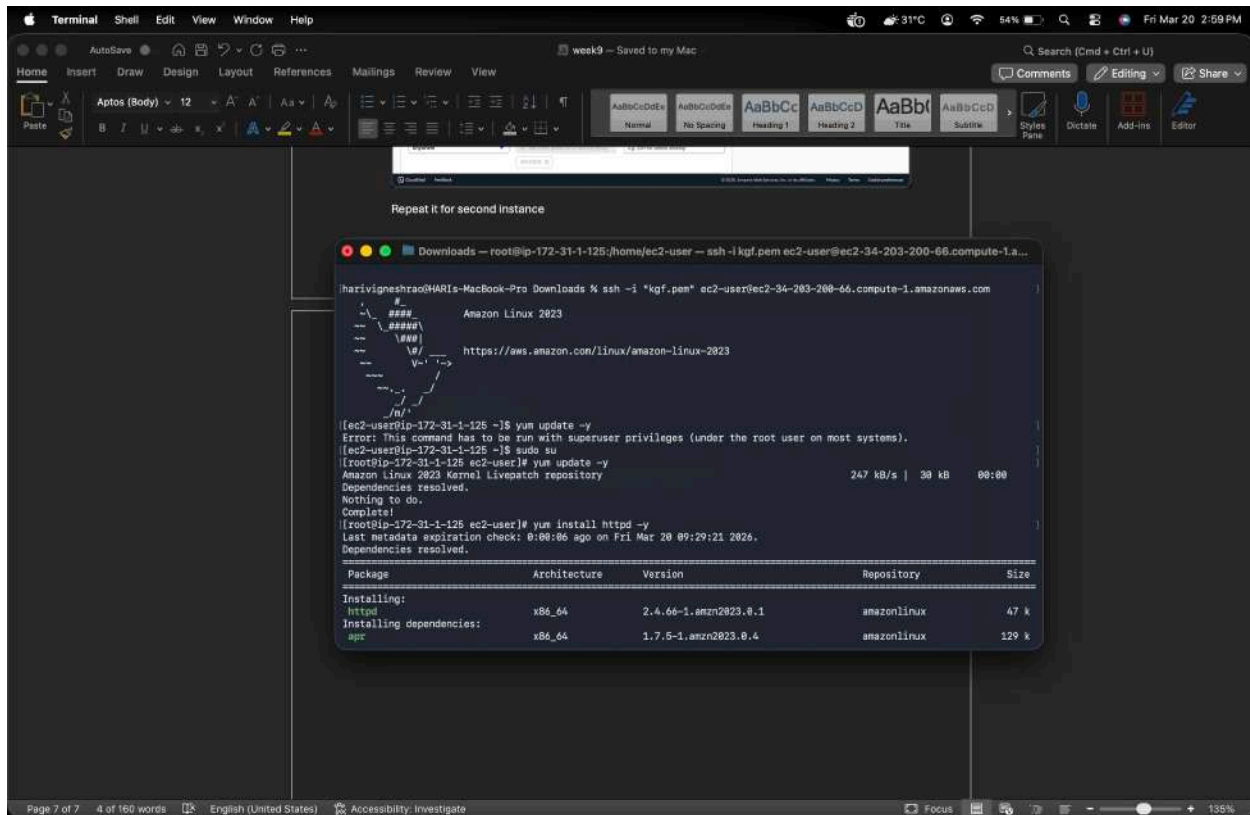


Now you create second **EC2 instances** running Apache, with message **This is Servier 2** each serving a simple web page.



Repeat it for second instance


```
yum update -y
yum install httpd -y
```



The screenshot shows a Mac screen with a terminal window open. The terminal window displays the output of the following commands:

```
Repeat it for second instance

[ec2-user@ip-172-31-1-125 ~]$ yum update -y
Error: This command has to be run with superuser privileges (under the root user on most systems).
[ec2-user@ip-172-31-1-125 ~]$ sudo su
[root@ip-172-31-1-125 ec2-user]# yum update -y
Amazon Linux 2023 Kernel Livepatch repository
Dependencies resolved.
Nothing to do.
Complete!
[root@ip-172-31-1-125 ec2-user]# yum install httpd -y
Last metadata expiration check: 0:00:06 ago on Fri Mar 20 09:21:21 2026.
Dependencies resolved.

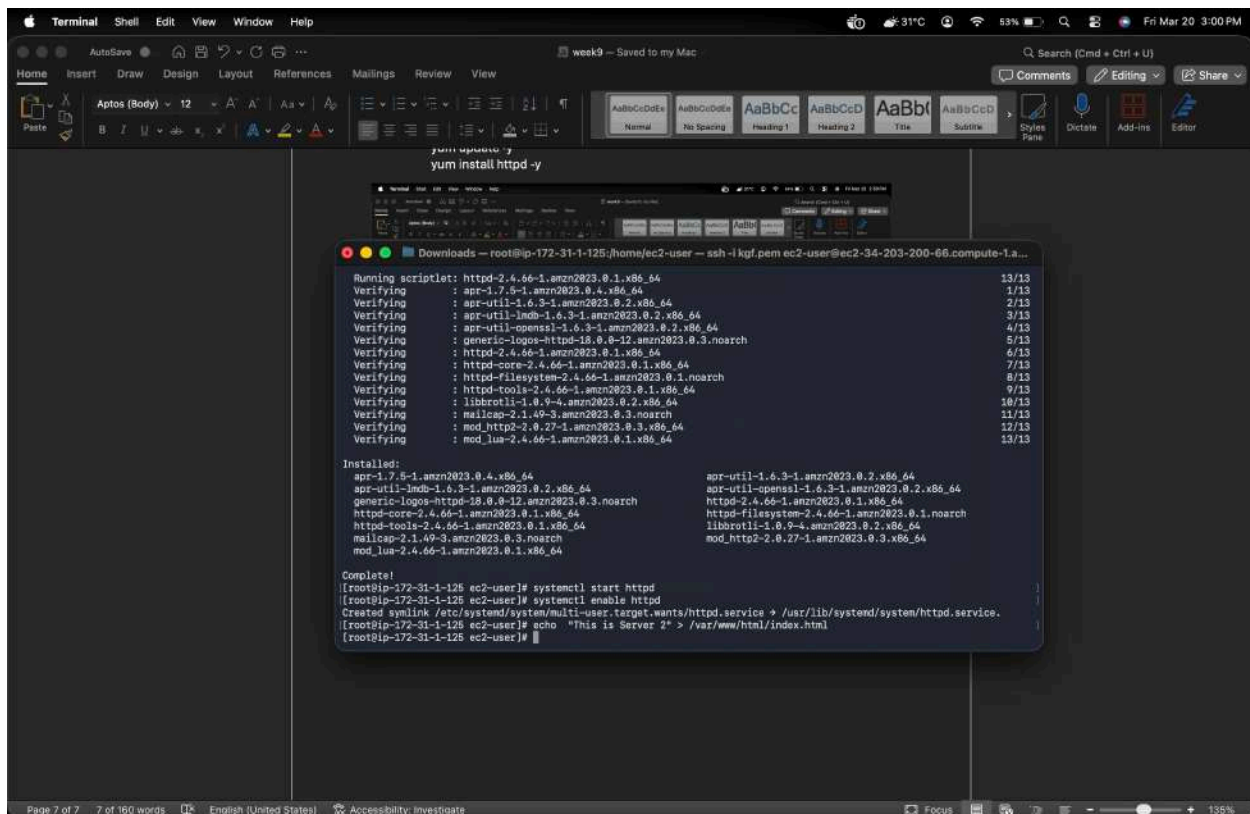
Package Architecture Version Repository Size
Installing:
httpd x86_64 2.4.66-1.amzn2023.0.1 amazonlinux 47 k
Installing dependencies:
apr x86_64 1.7.5-1.amzn2023.0.4 amazonlinux 129 k
```

The terminal window also shows the output of the following command:

```
harivigneshrao@HARIS-MacBook-Pro Downloads % ssh -i "kgf.pem" ec2-user@ec2-34-203-200-66.compute-1.a...
```

```
systemctl start httpd
systemctl enable httpd
```

```
echo "This is Server 2" > /var/www/html/index.html
```



```
Running scriptlet: httpd-2.4.66-1.amzn2023.0.1.x86_64 13/13
Verifying : apr-1.7.5-1.amzn2023.0.4.x86_64 1/13
Verifying : apr-util-1.6.3-1.amzn2023.0.2.x86_64 2/13
Verifying : apr-util-lmdb-1.6.3-1.amzn2023.0.2.x86_64 3/13
Verifying : apr-util-openssl-1.6.3-1.amzn2023.0.2.x86_64 4/13
Verifying : generic-logos-httpd-18.0.0-12.amzn2023.0.3.noarch 5/13
Verifying : httpd-2.4.66-1.amzn2023.0.1.x86_64 6/13
Verifying : httpd-core-2.4.66-1.amzn2023.0.1.x86_64 7/13
Verifying : httpd-filesystem-2.4.66-1.amzn2023.0.1.noarch 8/13
Verifying : httpd-tools-2.4.66-1.amzn2023.0.1.x86_64 9/13
Verifying : libbrotli-1.0.9-4.amzn2023.0.2.x86_64 10/13
Verifying : mailcap-2.1.49-3.amzn2023.0.3.noarch 11/13
Verifying : mod_http2-2.0.27-1.amzn2023.0.3.x86_64 12/13
Verifying : mod_lua-2.4.66-1.amzn2023.0.1.x86_64 13/13

Installed:
apr-1.7.5-1.amzn2023.0.4.x86_64          apr-util-1.6.3-1.amzn2023.0.2.x86_64
apr-util-lmdb-1.6.3-1.amzn2023.0.2.x86_64  apr-util-openssl-1.6.3-1.amzn2023.0.2.x86_64
generic-logos-httpd-18.0.0-12.amzn2023.0.3.noarch  httpd-2.4.66-1.amzn2023.0.1.x86_64
httpd-core-2.4.66-1.amzn2023.0.1.x86_64  httpd-filesystem-2.4.66-1.amzn2023.0.1.noarch
httpd-tools-2.4.66-1.amzn2023.0.1.x86_64  libbrotli-1.0.9-4.amzn2023.0.2.x86_64
mailcap-2.1.49-3.amzn2023.0.3.noarch      mod_http2-2.0.27-1.amzn2023.0.3.x86_64
mod_lua-2.4.66-1.amzn2023.0.1.x86_64

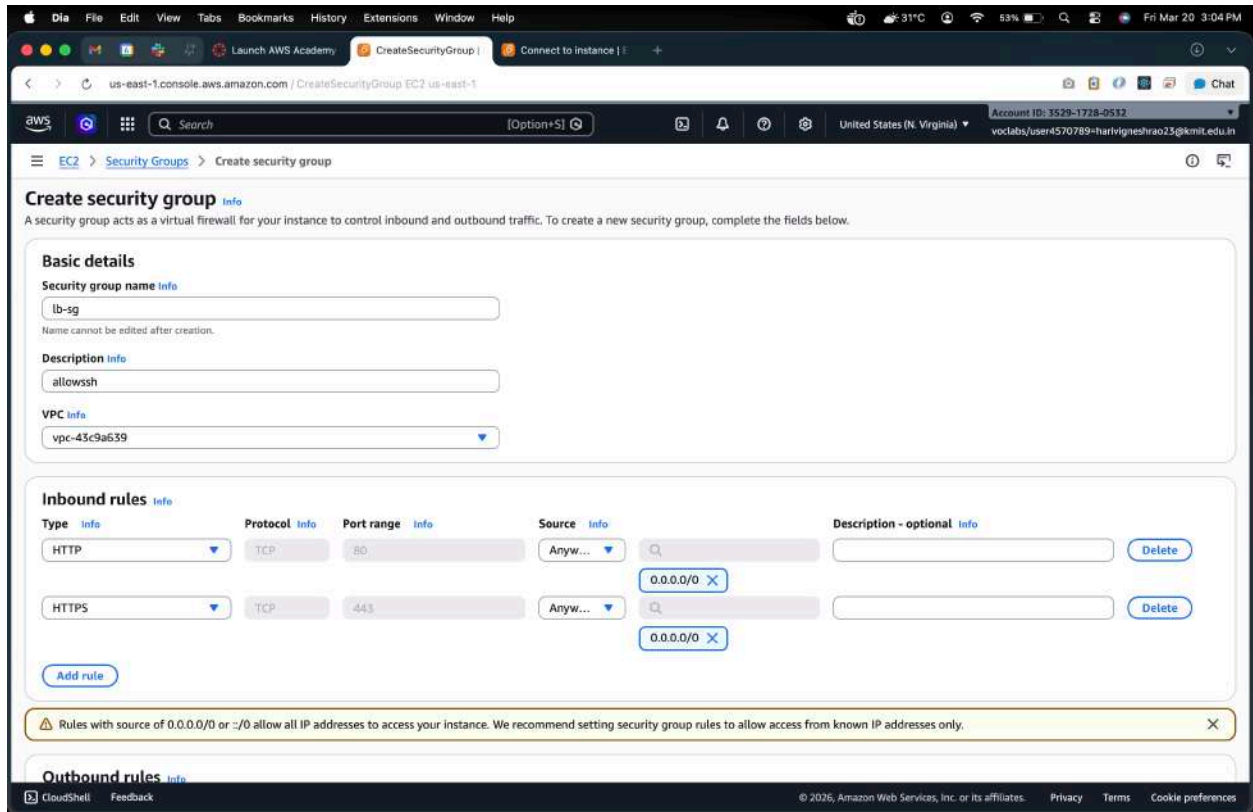
Complete!
[root@ip-172-31-1-125 ec2-user]# systemctl start httpd
[root@ip-172-31-1-125 ec2-user]# systemctl enable httpd
Created symlink /etc/systemd/system/multi-user.target.wants/httpd.service → /usr/lib/systemd/system/httpd.service.
[root@ip-172-31-1-125 ec2-user]# echo "This is Server 2" > /var/www/html/index.html
[root@ip-172-31-1-125 ec2-user]#
```

Step 2: Create a Load Balancer

AWS provides **Elastic Load Balancer (ELB)**. We will create an **Application Load Balancer (ALB)**.

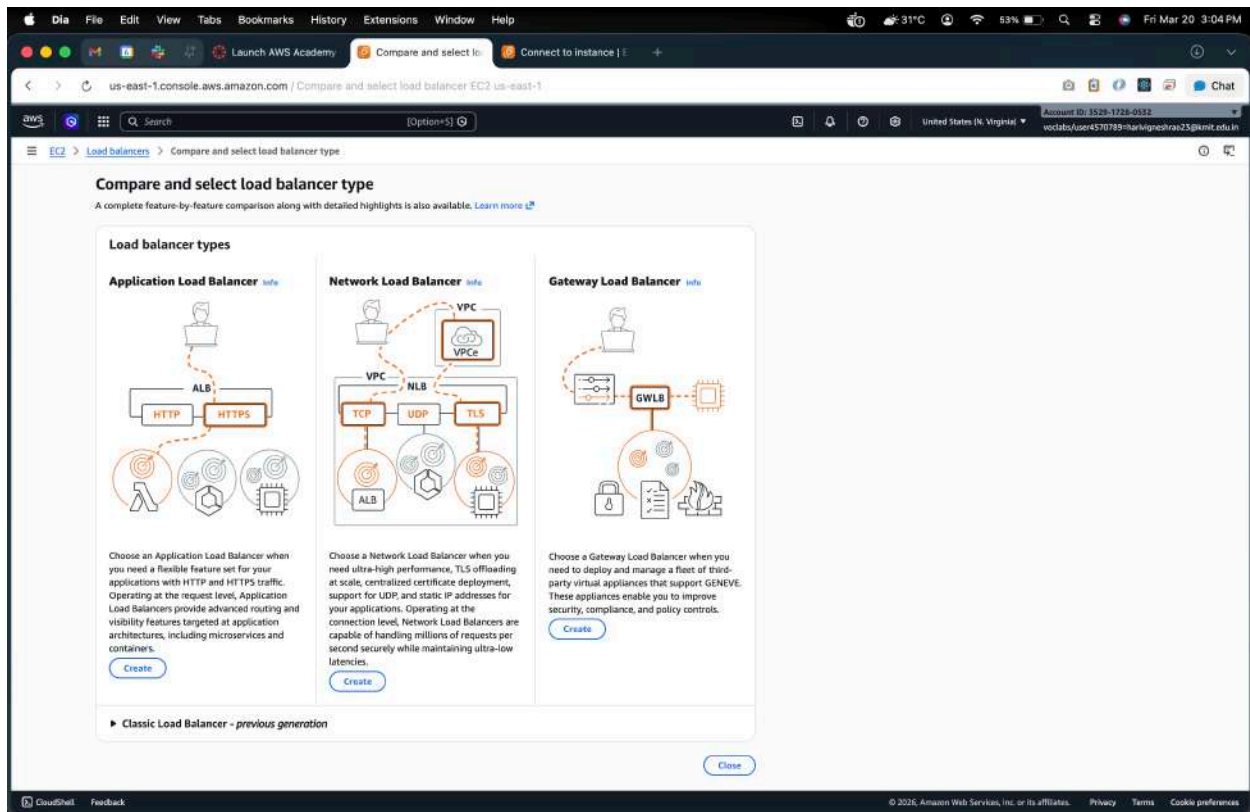
2.1 Configure Security Group for Load Balancer

- Go to **EC2 → Security Groups → Create Security Group**
 - Name: lb-sg
 - Allow **HTTP (80)** from anywhere
 - Optional: **HTTPS (443)** if using SSL



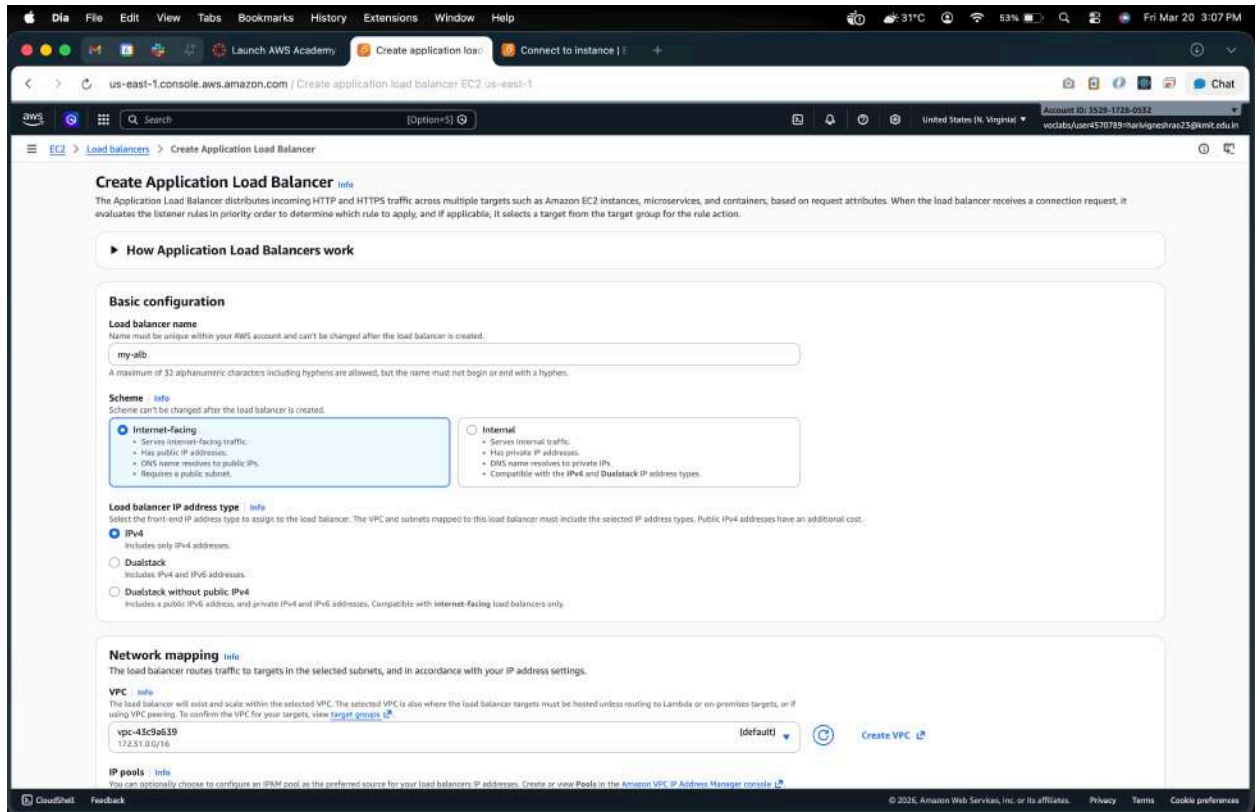
2.2 Create the Load Balancer

1. Go to **EC2** → **Load Balancers** → **Create Load Balancer** → **Application Load Balancer**



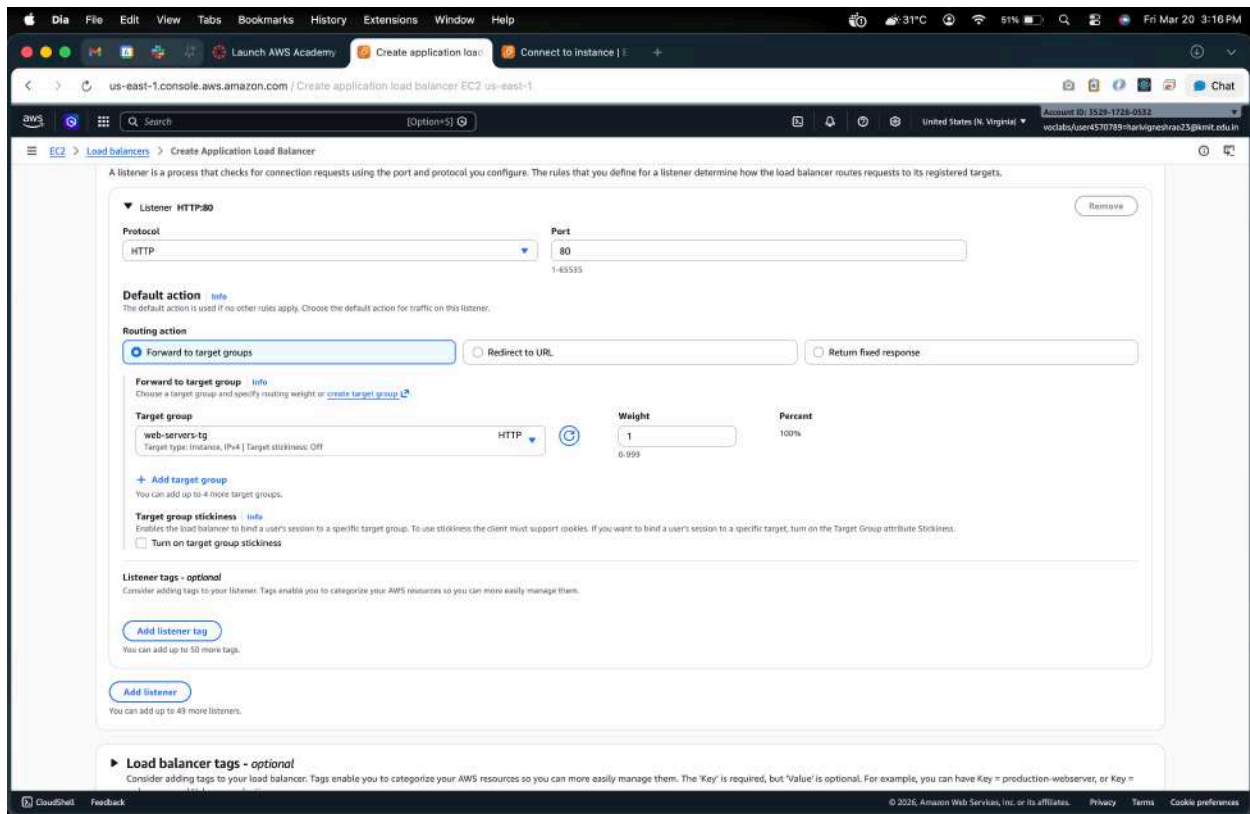
2. Basic Configuration:

- Name: my-alb
- Scheme: Internet-facing
- IP address type: IPv4



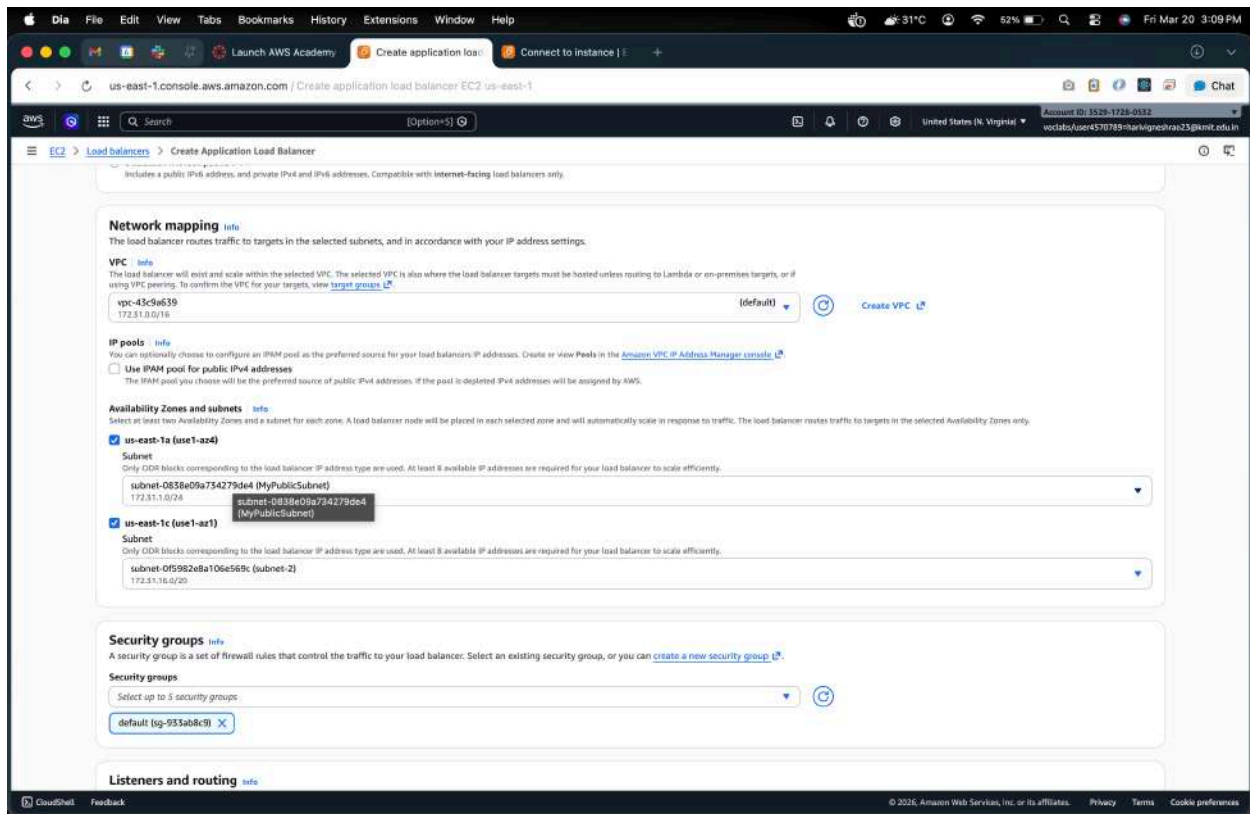
3. Listeners:

- Default: HTTP (80)



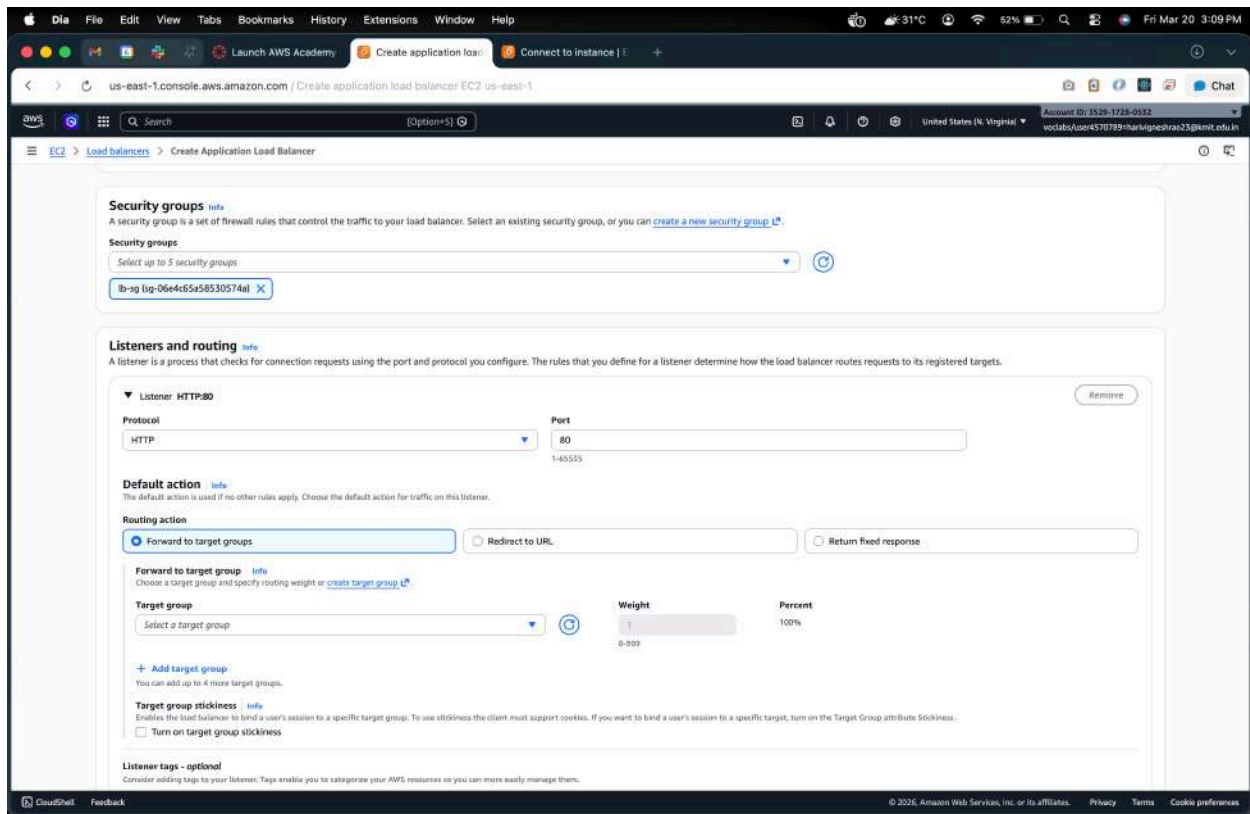
2.3 Configure Availability Zones

- Select the default **VPC** where your EC2 instances exist
- Select **two /all subnets** (one for each instance if possible)



2.4 Configure Security Groups

- Choose the **lb-sg** security group created earlier



2.5 Configure Target Groups

1. Create a new target group:

- Name: web-servers-tg
- Target type: Instance
- Protocol: HTTP
- Port: 80

Create target group

A target group can be made up of one or more targets. Your load balancer routes requests to the targets in a target group and performs health checks on the targets.

Settings - immutable
Choose a target type and the load balancer and listener will route traffic to your target. These settings can't be modified after target group creation.

Target type
Indicate what resource type you want to target. Only the selected resource type can be registered to this target group.

- ☒ **Instances**
Supports load balancing to instances in a VPC. Integrate with Auto Scaling Groups or ECS services for automatic management.
Suitable for: ALB, NLB, GWLB
- ☐ **IP addresses**
Supports load balancing to VPC and on-premises resources. Facilitates routing to IP addresses and network interfaces on the same instance. Supports IPv6 targets.
Suitable for: ALB, NLB, GWLB
- ☐ **Lambda function**
Supports load balancing to a single Lambda function. ALB required as traffic source.
Suitable for: ALB
- ☐ **Application Load Balancer**
Allows use of static IP addresses and PrivateLink with an Application Load Balancer. NLB required as traffic source.
Suitable for: NLB

Target group name
Name must be unique per Region per AWS account.
web-servers-tg
Accepts: a-z, A-Z, 0-9, and hyphen (-). Can't begin or end with hyphen. 1-32 total characters. Count: 14/32

Protocol
Protocol for communication between the load balancer and targets.
HTTP

Port
Port number where targets receive traffic. Can be overridden for individual targets during registration.
80
1-65535

IP address type
Only targets with the indicated IP address type can be registered to this target group.

- ☒ **IPv4**
Each instance has a default network interface (eth0) that is assigned the primary private IPv4 address. The instance's primary private IPv4 address is the one that will be applied to the target.
- ☐ **IPv6**
Each instance you register must have an assigned primary IPv6 address. This is configured on the instance's default network interface (eth0). [Learn more](#)

VPC
Select the VPC with the instances that you want to include in the target group. Only VPCs that support the IP address type selected above are available in this list.

- Health checks: / (default)

Health checks
The associated load balancer periodically sends requests, per the settings below, to the registered targets to test their status.

Health check protocol
HTTP

Health check path
Use the default path of "/" to perform health checks on the root, or specify a custom path if preferred.
/
Up to 1024 characters allowed.

Advanced health check settings

Target optimizer - optional
Use a target control port when the target has a strict concurrency limit.

Target control port
The port on which the target communicates its capacity. This value can't be modified after target group creation.
Enter target control port
Valid range: 1-65535

Attributes
Certain default attributes will be applied to your target group. You can view and edit them after creating the target group.

Tags - optional
Consider adding tags to your target group. Tags enable you to categorize your AWS resources so you can more easily manage them.
No tags associated with this resource.
Add new tag
You can add up to 30 tags.

Cancel Next

2. Register Targets:

- Select EC2 INSTANCES HERE --- **webserver-1** and **webserver-2**
- Click **Include as pending** → **Register**

The screenshot shows the AWS Management Console interface for creating a target group. The left sidebar indicates the current step is 'Register targets'. The main content area is titled 'Register targets - recommended' and shows a list of available EC2 instances. Two instances, 'webserver-1' and 'webserver-2', are selected. Below the list, the 'Include as pending' button is visible. The 'Review targets' section at the bottom shows the selected instances and their details.

Available instances (2)

Instance ID	Name	State	Security groups	Zone	Private IPv4 address
i-0dc268ed5ea755dde	webserver-2	Running	launch-wizard-38	us-east-1a	172.31.1.125
i-00c2c692fabba3005	webserver-1	Running	launch-wizard-37	us-east-1a	172.31.1.87

0 selected

Ports for the selected instances:

Ports for routing traffic to the selected instances:

80

1-65535 (separate multiple ports with commas)

Include as pending below

Review targets

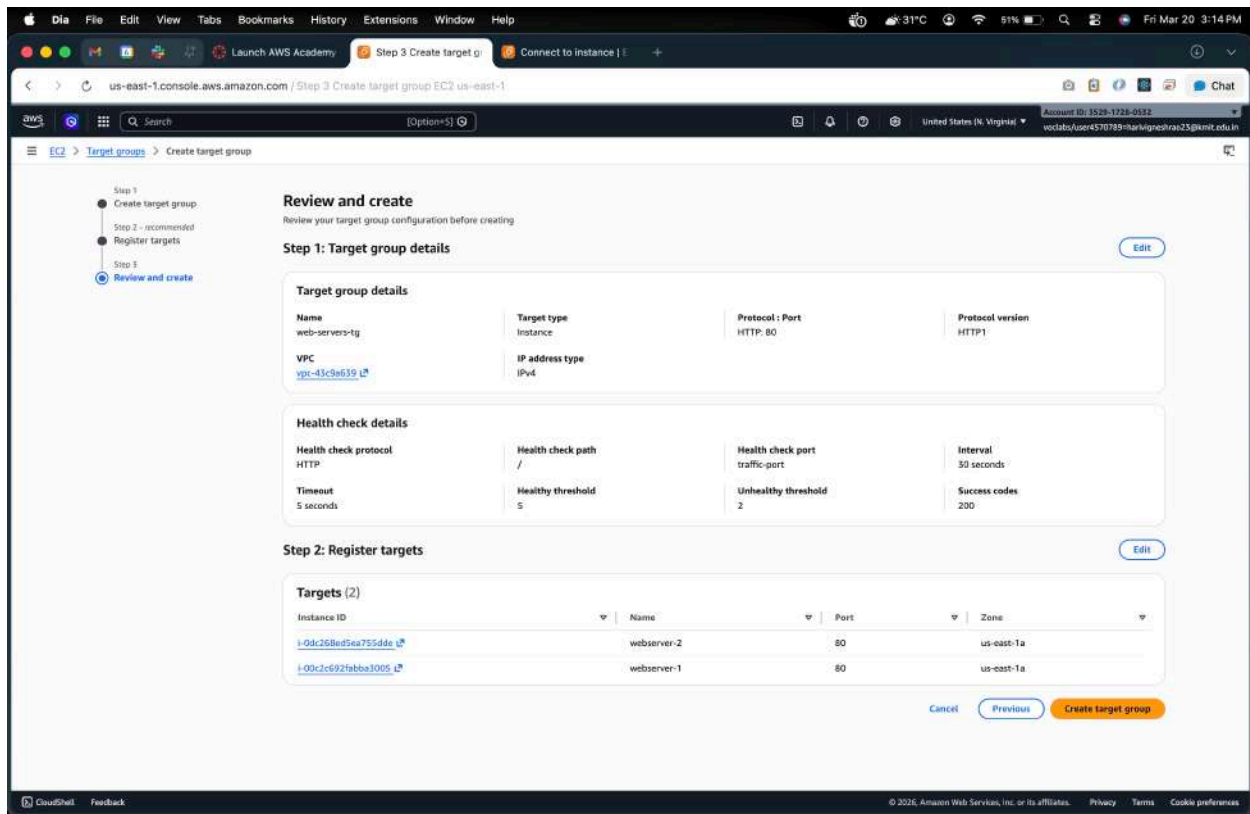
Targets (2)

Show only pending

Instance ID	Name	Port	State	Security groups	Zone	Private IPv4 address	Subnet ID	Launch time
i-0dc268ed5ea755dde	webserver-2	80	Running	launch-wizard-38	us-east-1a	172.31.1.125	subnet-0838e09a734279de4	March 20, 2026, 14:57 (UTC+05:30)
i-00c2c692fabba3005	webserver-1	80	Running	launch-wizard-37	us-east-1a	172.31.1.87	subnet-0838e09a734279de4	March 20, 2026, 14:52 (UTC+05:30)

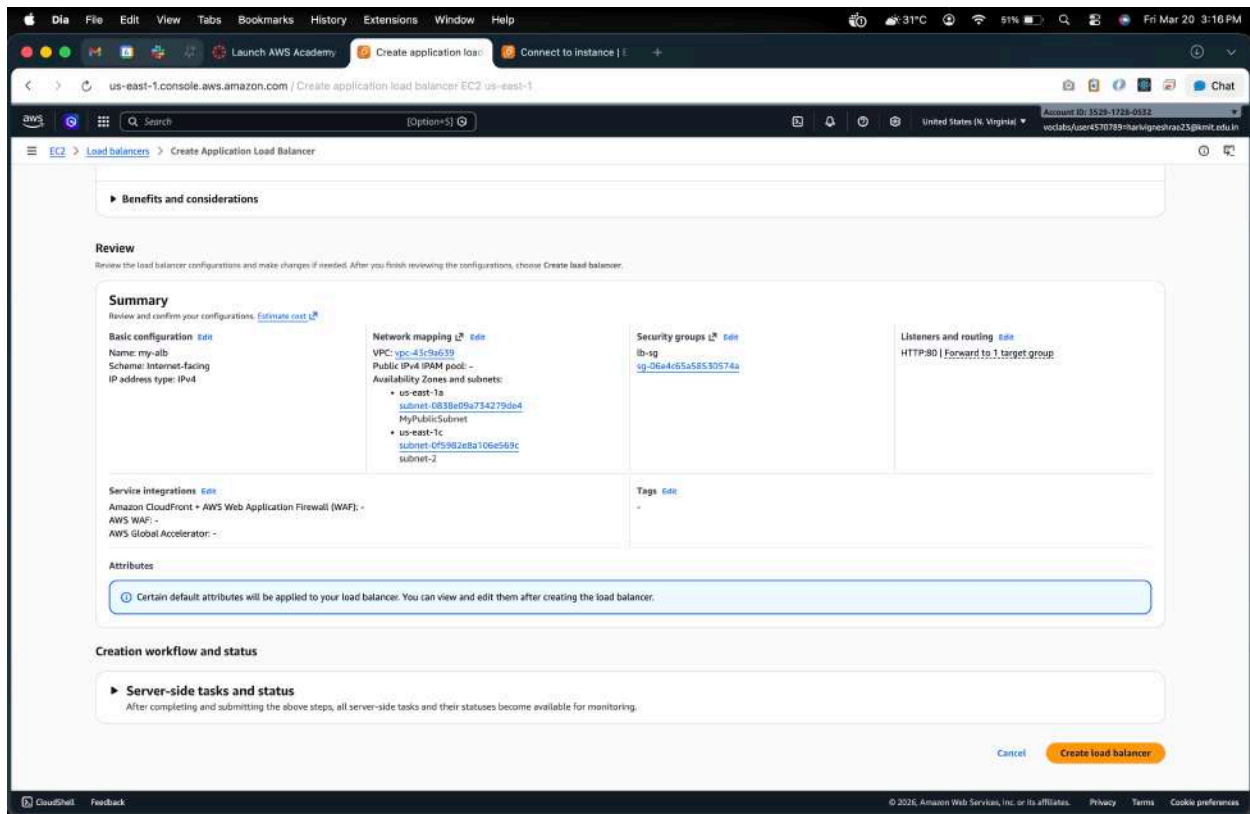
2 pending

Cancel Previous Next



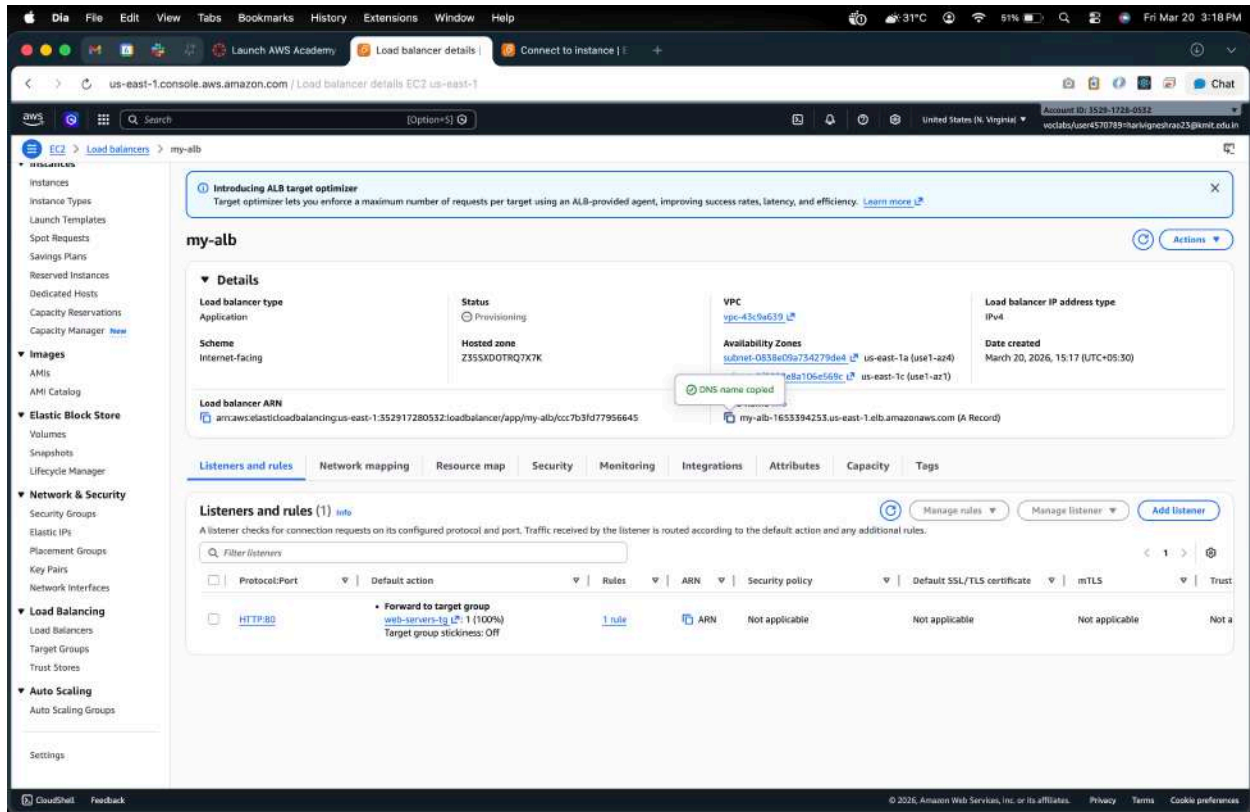
2.6 Review and Create

- Review all settings → Click **Create Load Balancer**
- AWS will take a few minutes to provision the ALB

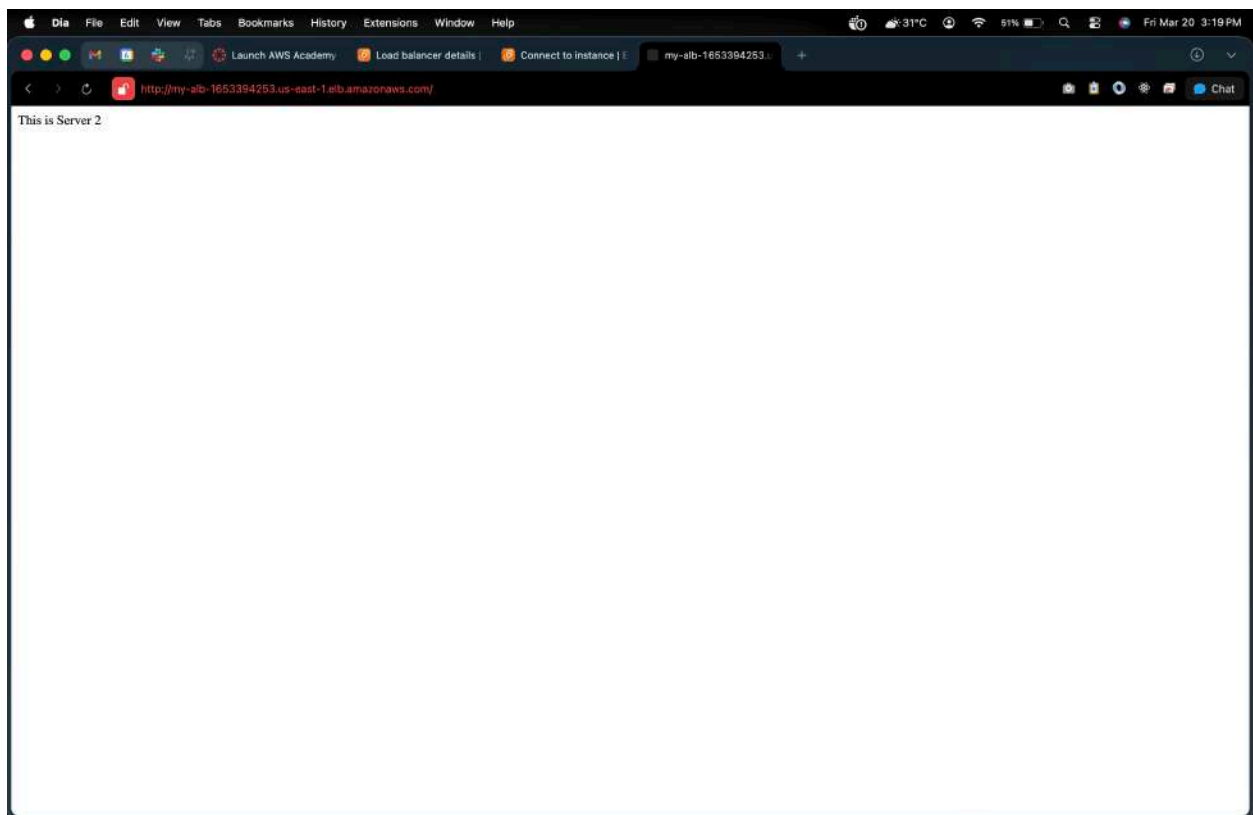
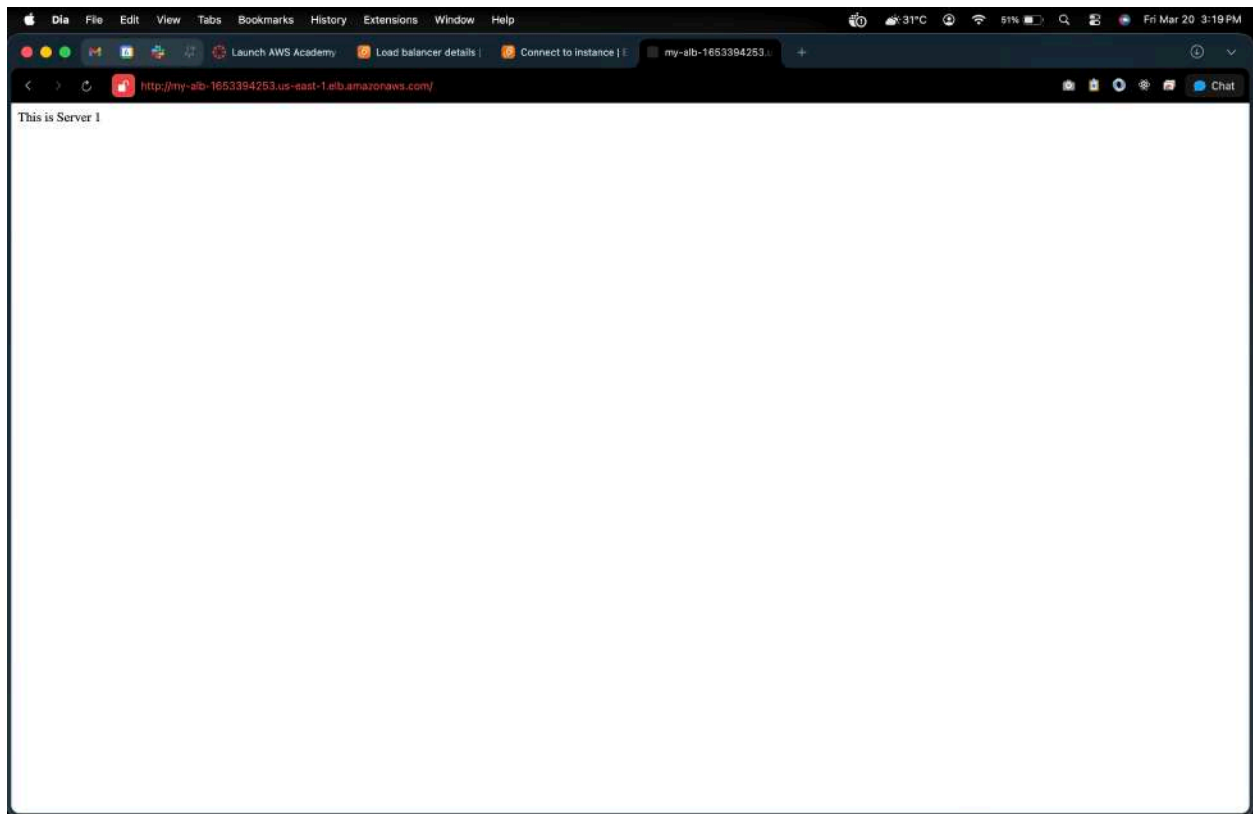


Step 3: Verify Load Balancer

1. Go to **Load Balancers** → **Select ALB** → **Description** → **DNS name**
2. Copy the **DNS name** (something like my-alb-123456.us-east-1.elb.amazonaws.com)



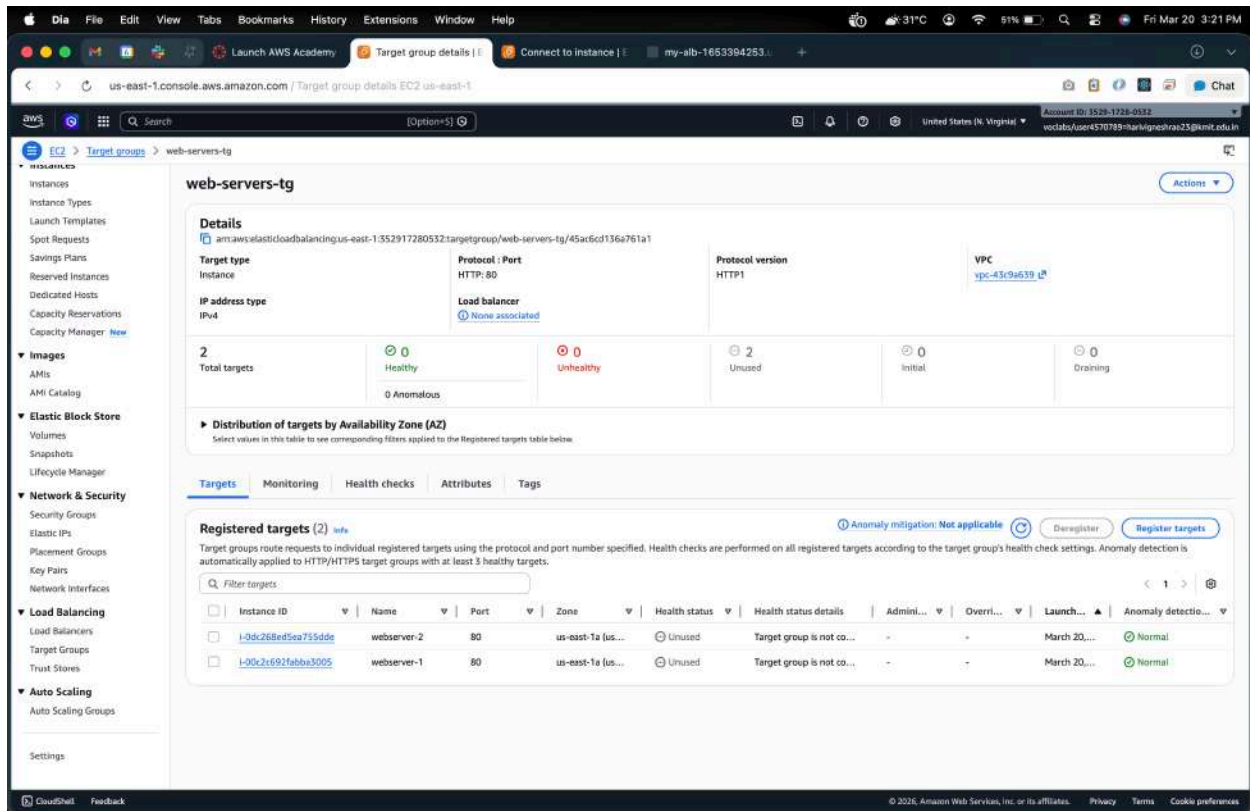
3. Paste it in a browser → You should see either **WebServer-1** or **WebServer-2** page.
 - Refresh multiple times → Load balancer distributes traffic between the two instances



Step 4: Optional – Test Health Checks

- Go to **Target Groups** → **web-servers-tg** → **Targets**
- Ensure **Status** of both instances is healthy

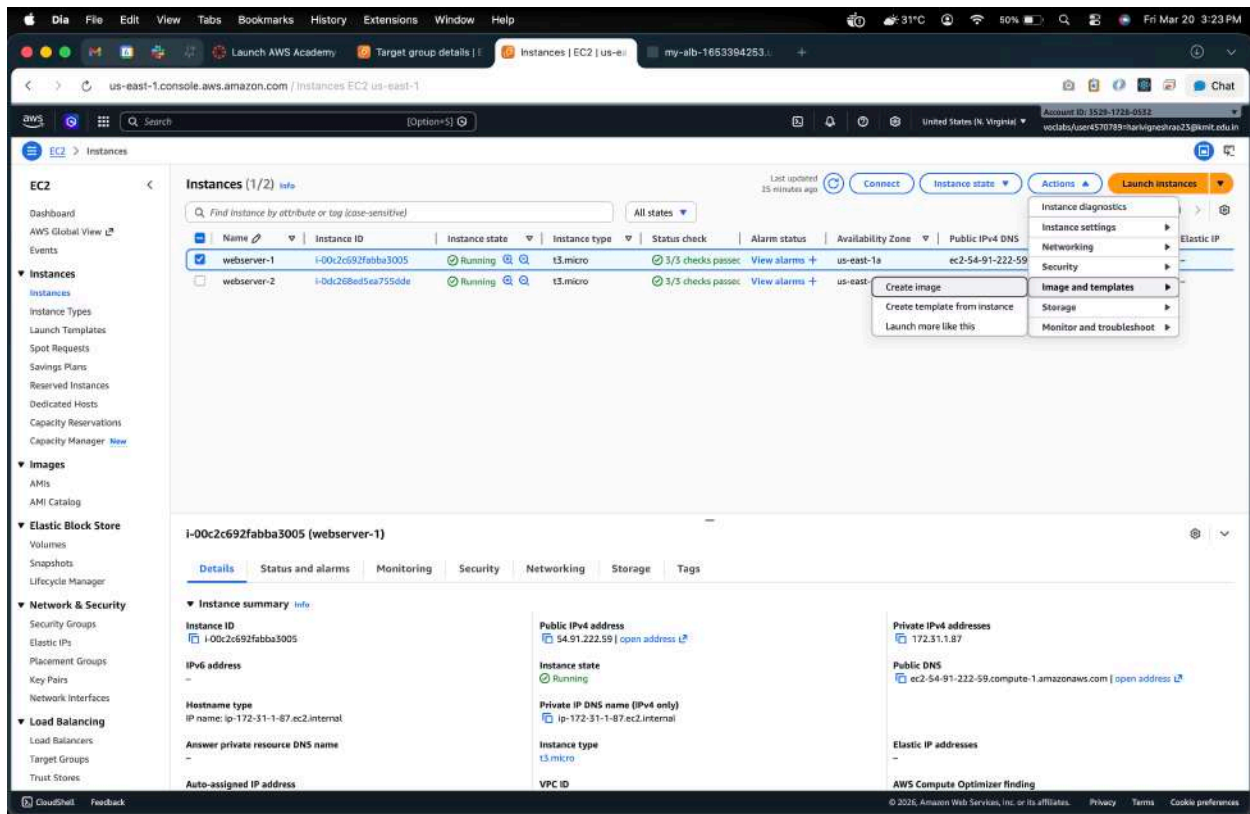
If a server is unhealthy, check the bootstrapping script or security group rules.



Step-by-Step: Auto Scaling Group with Load Balancer

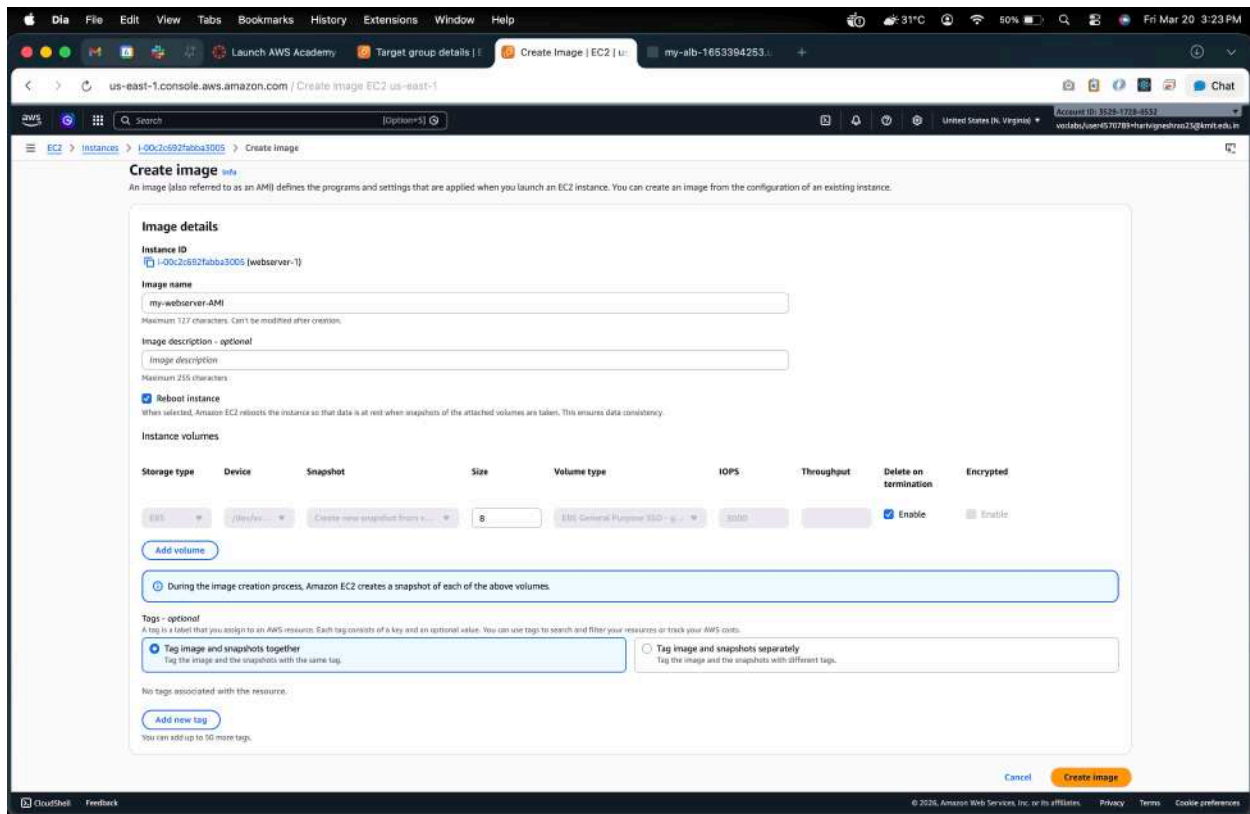
Step 1: Create an AMI from Existing EC2 Instance

1. Go to **EC2 Console** → **Instances**
2. Select the EC2 instance you already configured with application
3. Click **Actions** → **Image and Templates** → **Create Image**



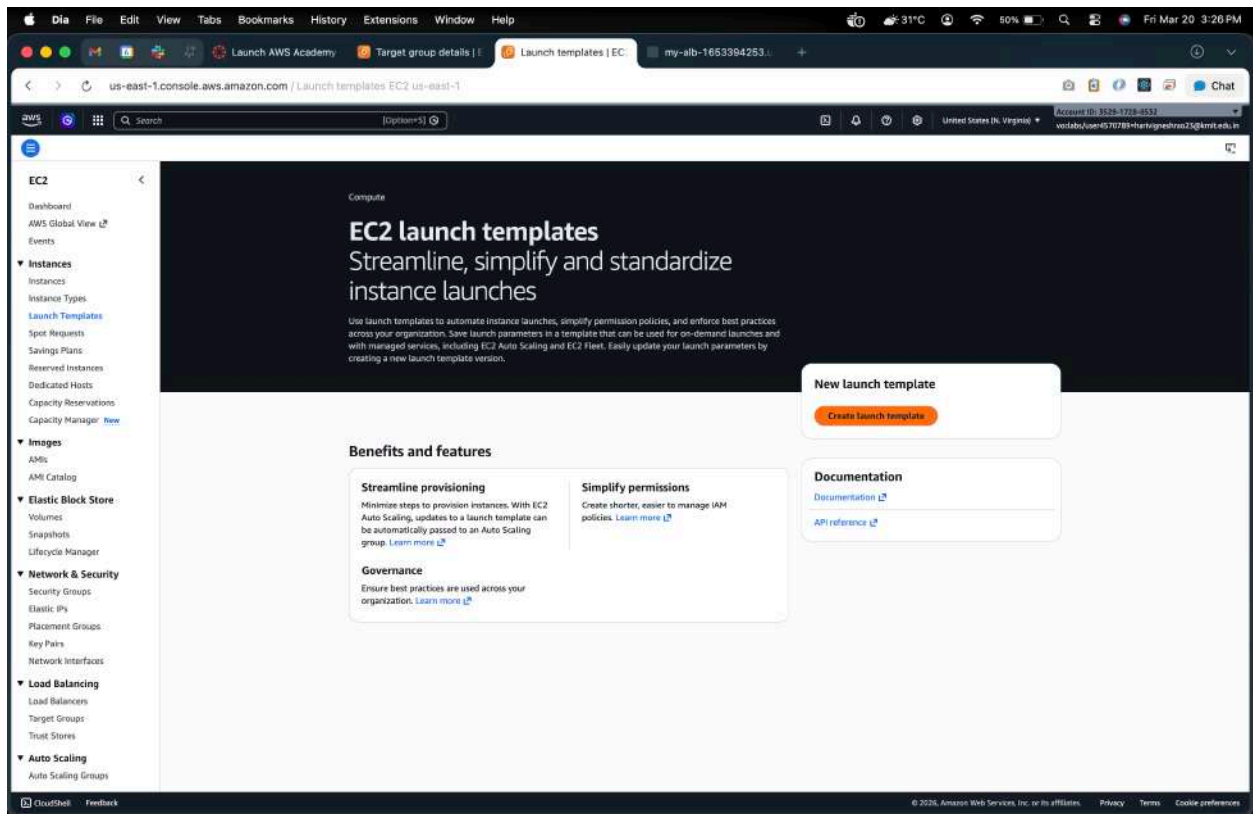
4. **Provide Image Name:** e.g., my-webserver-AMI
5. Optional: Add description
6. Click **Create Image**
7. Wait for the AMI to become **available** (status: available in **AMIs** section)

This AMI will be used for launching new instances automatically in ASG.

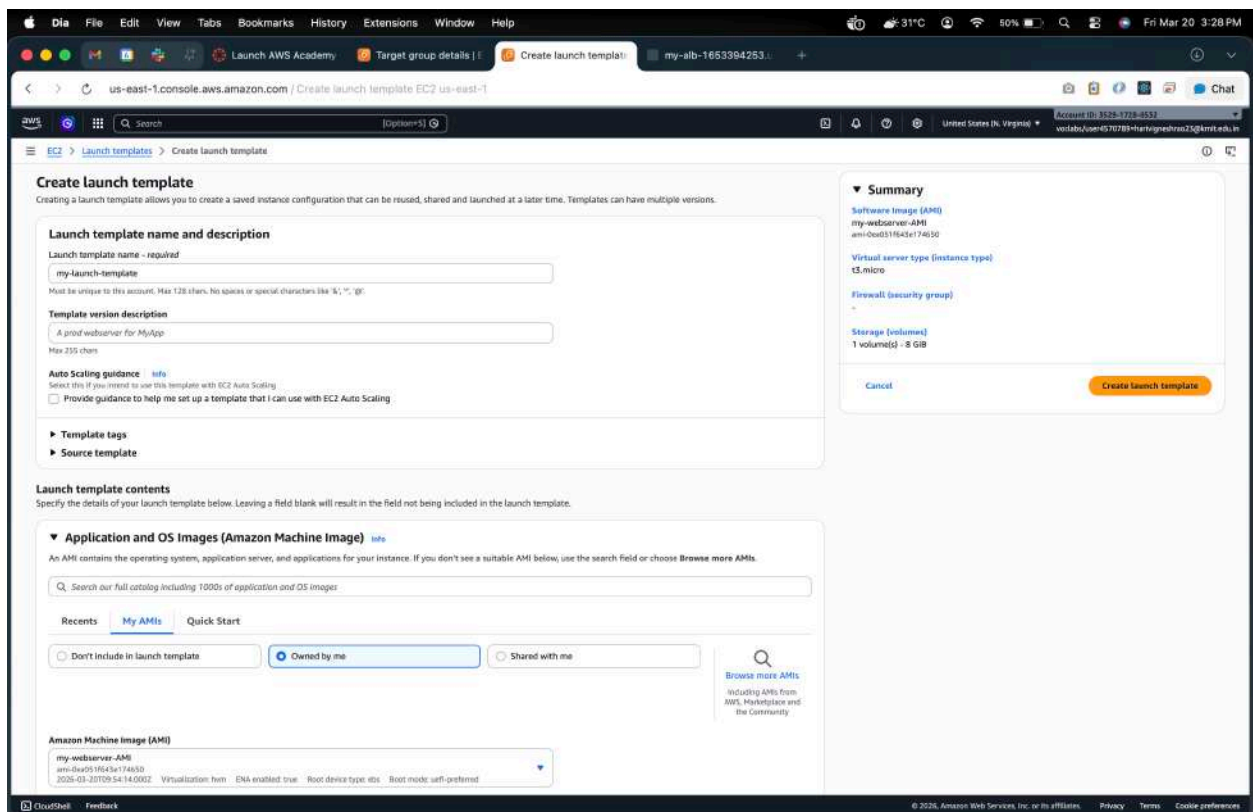


Step 2: Create a Launch Template

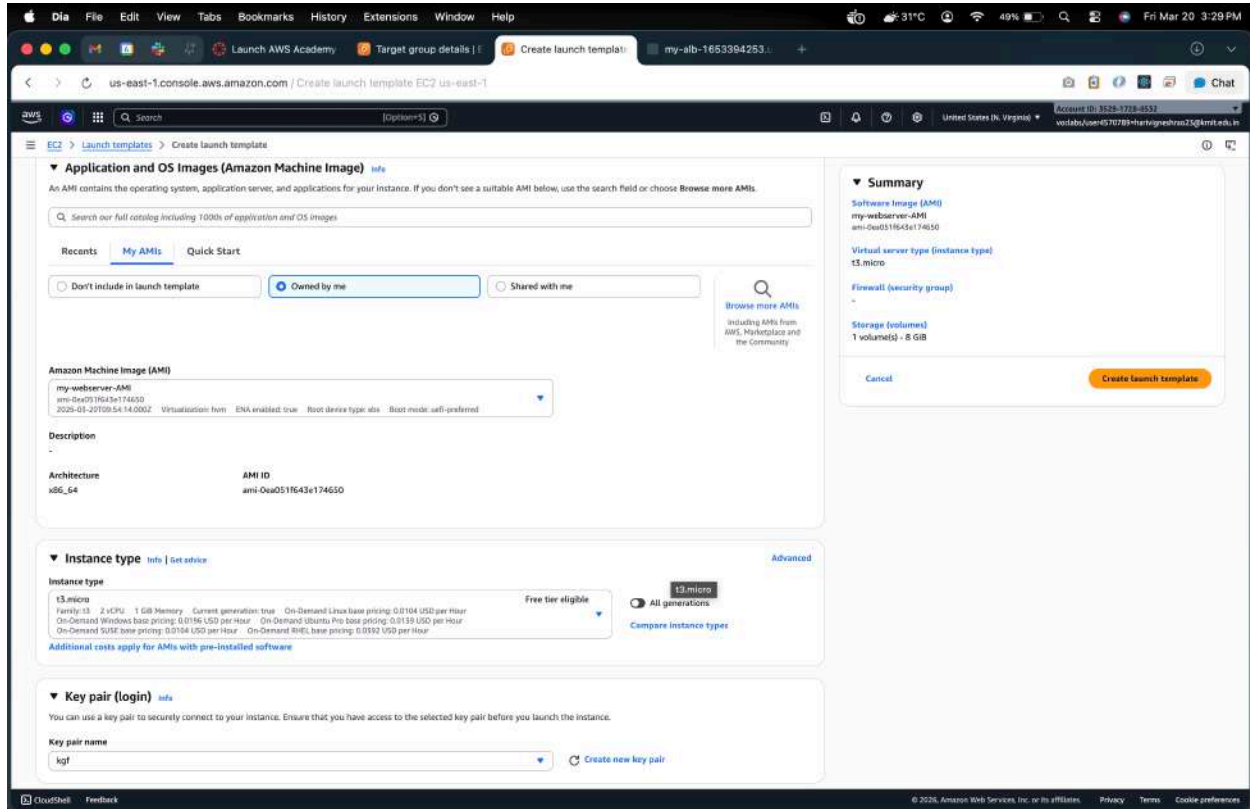
1. Go to **EC2** → **instances** → **Launch Templates** → **Create Launch Template**



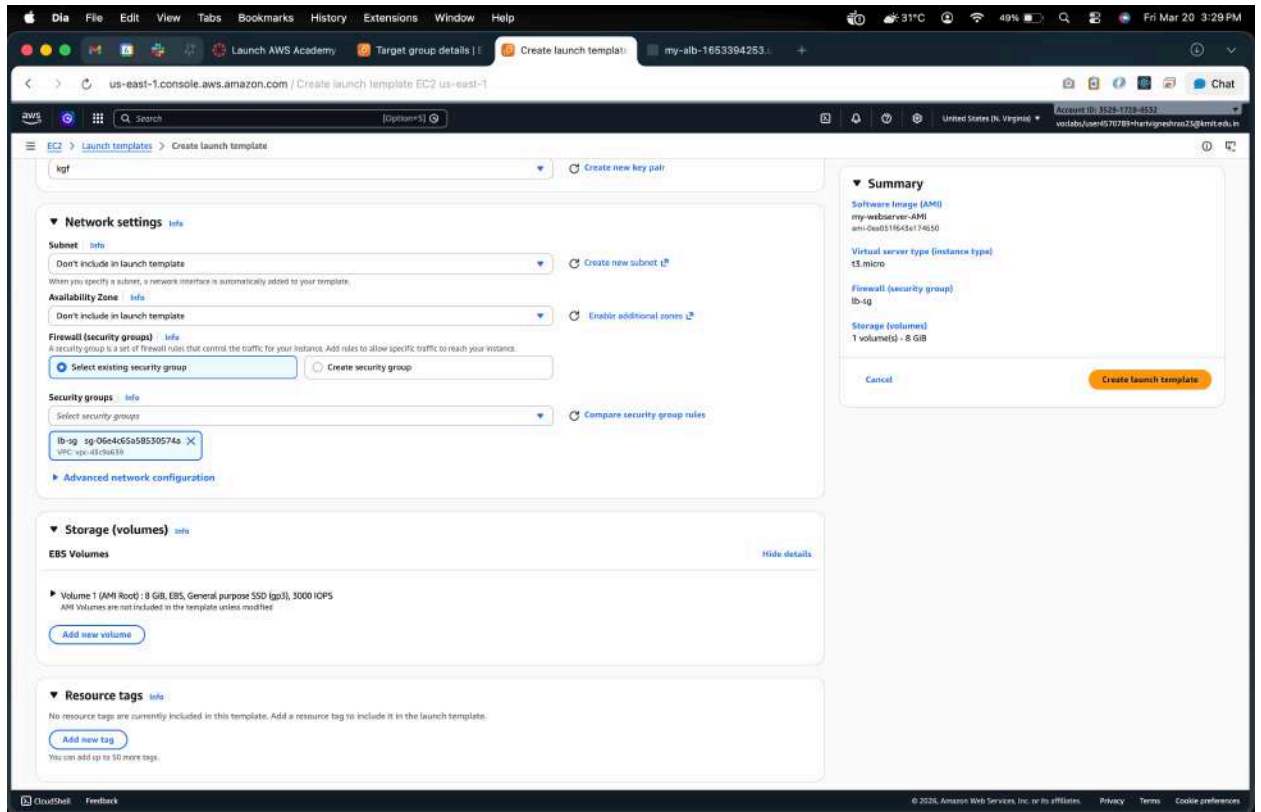
2. Launch Template Name: my-launch-template



3. **AMI:** Select the AMI created in Step 1 (my-webserver-AMI)
4. **Instance Type:** t2.micro (or your choice)
5. **Key Pair:** Select your existing key pair



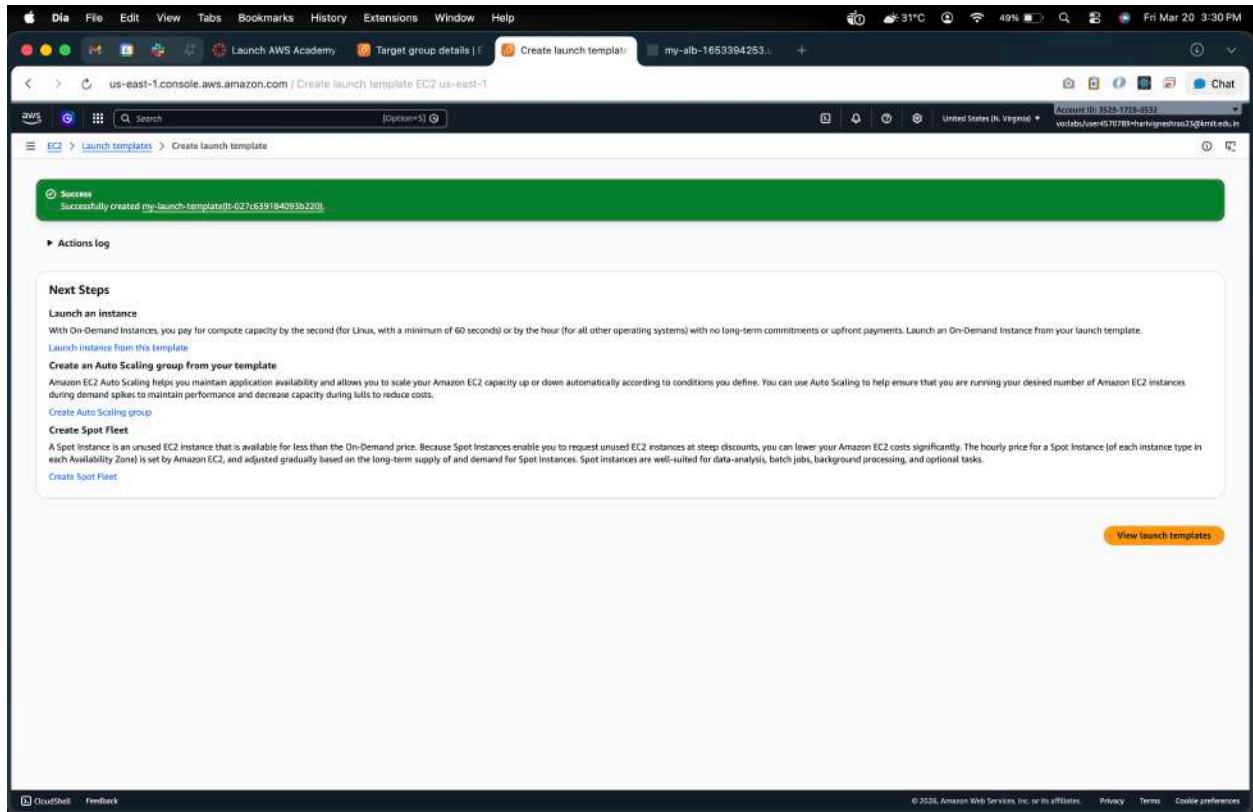
6. **Security Groups:** Select the **Load Balancer Security Group** (allow HTTP 80)



7. **User Data:** Optional (already baked into AMI)

8. Click **Create Launch Template**

Launch Template is ready for Auto Scaling.



Step 3: Create Auto Scaling Group (ASG)

1. Go to **EC2 → Auto Scaling Groups → Create Auto Scaling Group**

Amazon EC2 Auto Scaling
helps maintain the availability of your applications

Auto Scaling groups are collections of Amazon EC2 instances that enable automatic scaling and fleet management features. These features help you maintain the health and availability of your applications.

Create Auto Scaling group

Get started with EC2 Auto Scaling by creating an Auto Scaling group.

[Create Auto Scaling group](#)

How it works

An Auto Scaling group is a collection of Amazon EC2 instances that are treated as a logical unit. You configure settings for a group and its instances as well as define the group's minimum, maximum, and desired capacity. Setting different minimum and maximum capacity values forms the bounds of the group, which allows the group to scale as the load on your application spikes higher or lower, based on demand. To scale the Auto Scaling group, you can either make manual adjustments to the desired capacity or let Amazon EC2 Auto Scaling automatically add and remove capacity to meet changes in demand.

When launching fleets of instances, you can specify what percentage of your capacity should be fulfilled by On-Demand instances, and what percentage with Spot instances, to save up to 90% on EC2 costs. Amazon EC2 Auto Scaling lets you provision and balance capacity across Availability Zones to optimize availability. It also provides lifecycle hooks, instance health checks, and scheduled scaling to automate capacity management.

Pricing

Amazon EC2 Auto Scaling features have no additional fees beyond the service fees for Amazon EC2, CloudWatch (for scaling policies), and the other AWS resources that you use. Visit the pricing page of each service to learn more.

Getting started

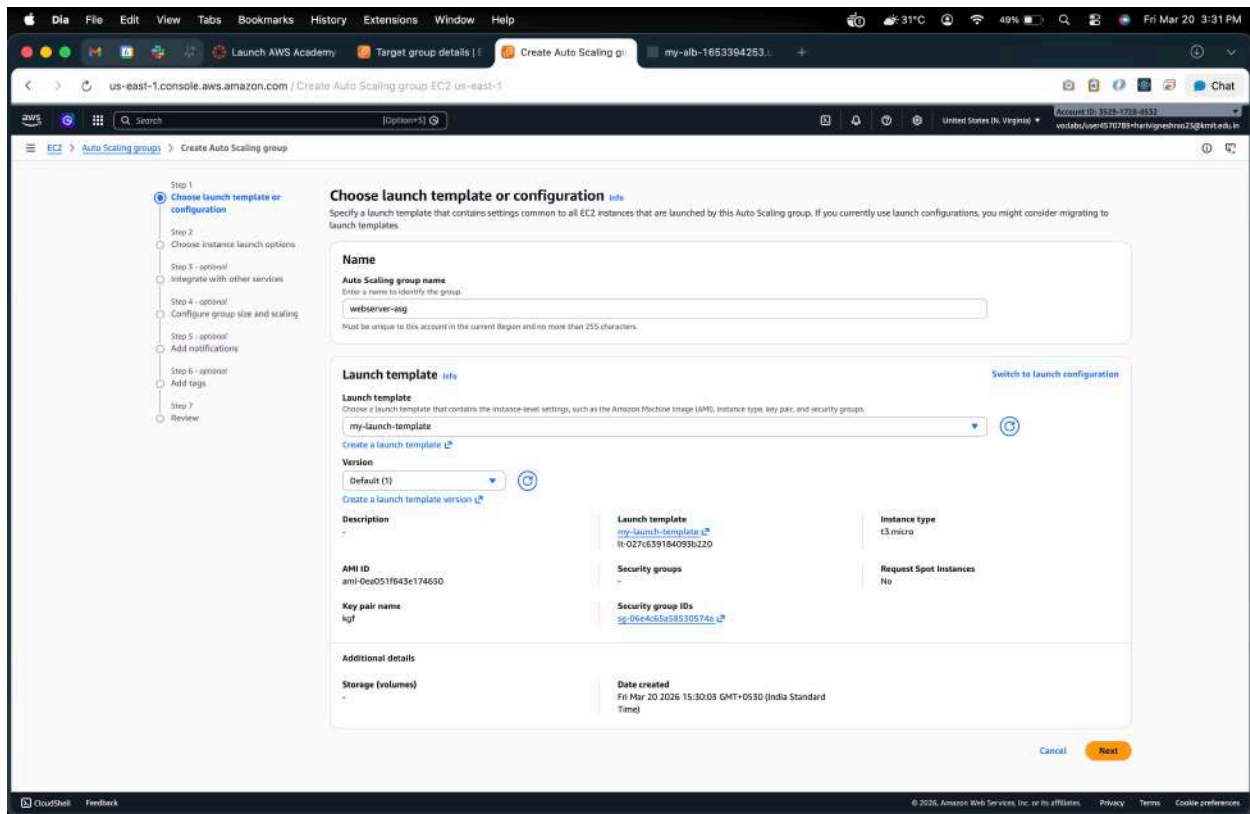
[What is Amazon EC2 Auto Scaling?](#)

[Getting started with Amazon EC2 Auto Scaling](#)

[Set up a scaled and load-balanced application](#)

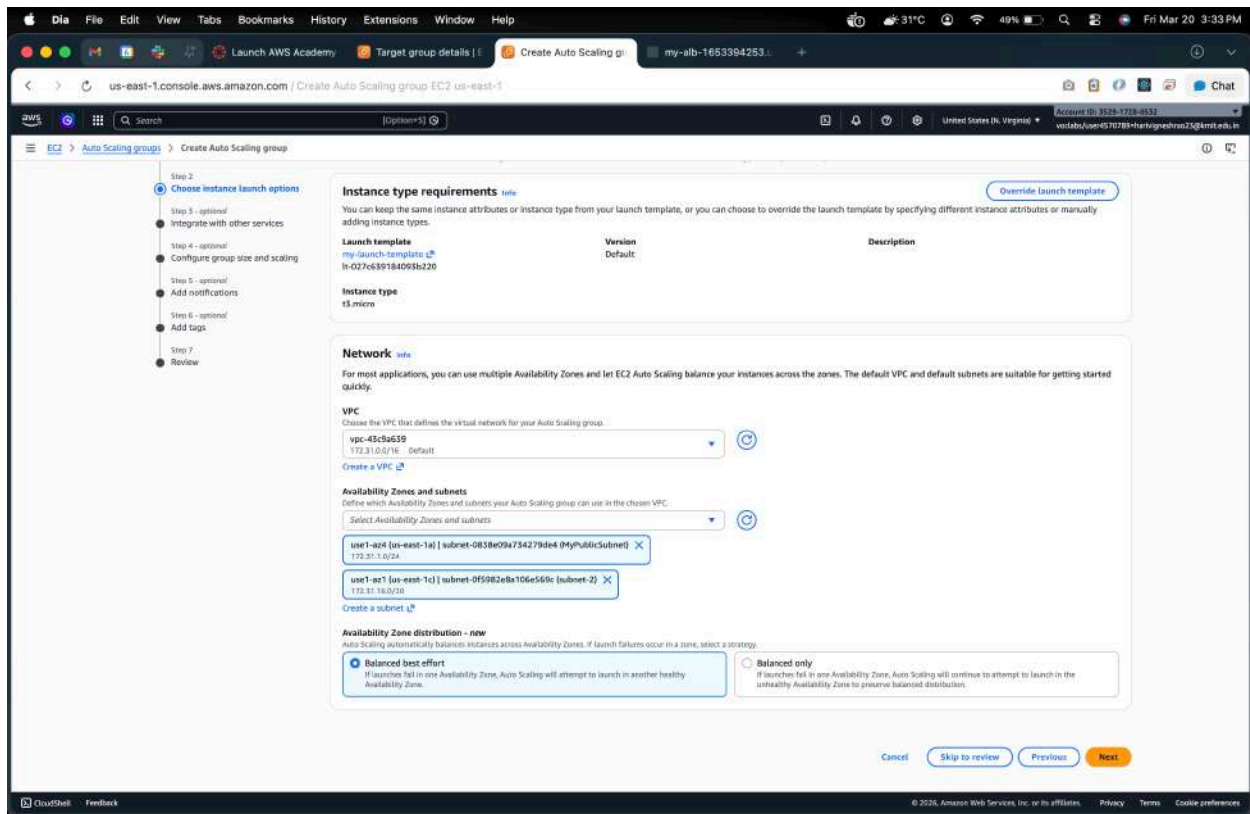
[FAQ](#)

2. **Name:** webserver-asg
3. **Launch Template:** Select my-launch-template
4. **Template Version:** Select **Version 1**
5. Click **Next**



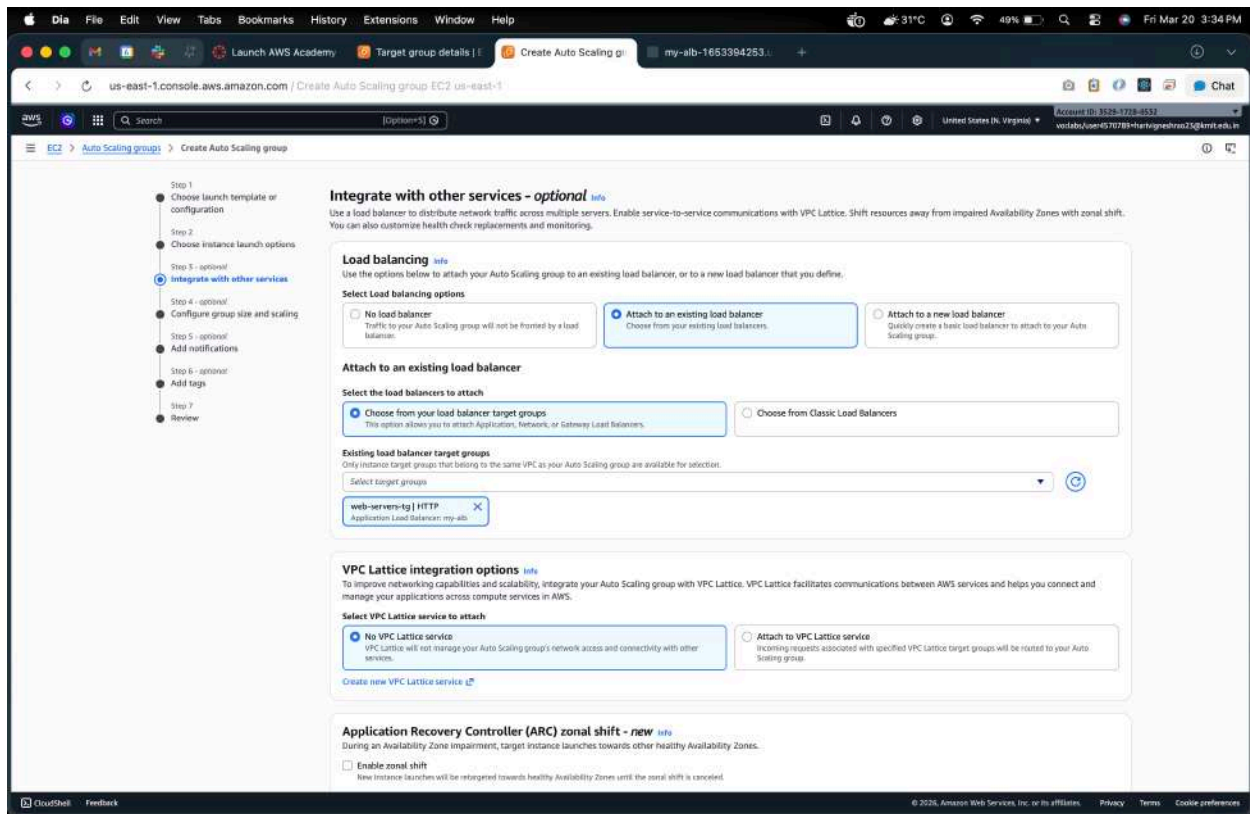
Step 3.1: Network Settings

1. **VPC:** Default VPC (or your existing VPC)
2. **Availability Zones (AZs):** Select **all** subnets (for high availability)



3. Load Balancer: Enable **Attach** to an existing load balancer

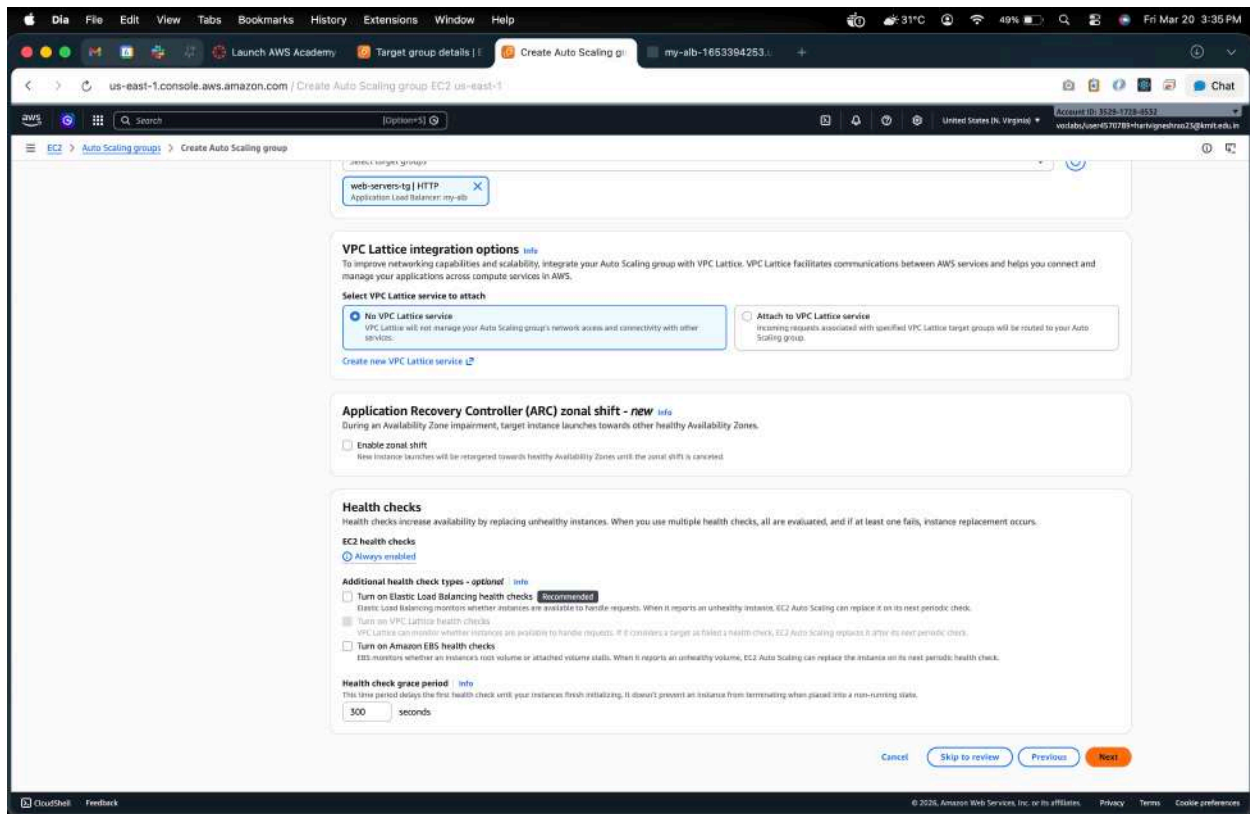
- Select your **existing Load Balancer**



4. Health Check: HTTP

- Set **Health Check Grace Period**: 300 seconds

5. Click **Next**



Step 3.2: Configure Group Size

1. **Desired Capacity:** 2 (initial instances)
2. **Minimum Capacity:** 1
3. **Maximum Capacity:** 4

Configure group size and scaling - optional [info](#)

Define your group's desired capacity and scaling limits. You can optionally add automatic scaling to adjust the size of your group.

Group size [info](#)

Set the initial size of the Auto Scaling group. After creating the group, you can change its size to meet demand, either manually or by using automatic scaling.

Desired capacity type

Choose the unit of measurement for the desired capacity value. vCPUs and MemoryGB are only supported for mixed instances groups configured with a set of instance attributes.

Units (number of instances)

Desired capacity

Specify your group size.

2

Scaling [info](#)

You can resize your Auto Scaling group manually or automatically to meet changes in demand.

Scaling limits

Set limits on how much your desired capacity can be increased or decreased.

Min desired capacity

1

Equal or less than desired capacity

Max desired capacity

4

Equal or greater than desired capacity

Automatic scaling - optional

Choose whether to use a target tracking policy. [info](#)

You can set up other metric-based scaling policies and scheduled scaling after creating your Auto Scaling group.

☒ **No scaling policies**

Your Auto Scaling group will remain at its initial size and will not dynamically resize to meet demand.

☐ **Target tracking scaling policy**

Choose a CloudWatch metric and target value and let the scaling policy adjust the desired capacity in proportion to the metric's value.

Instance maintenance policy [info](#)

Control your Auto Scaling group's availability during instance replacement events. This includes health checks, instance refreshes, maximum instance lifetime features and events that happen automatically to keep your group balanced, called rebalancing events.

Choose a replacement behavior depending on your availability requirements

Mixed behavior

☒ **No policy**

For rebalancing events, new instances will launch before terminating others. For all other events, instances terminate and launch at the same time.

Maximize availability

☐ **Launch before terminating**

Launch new instances and wait for them to be ready before terminating others. This allows you to go above your desired capacity by a given percentage and may temporarily increase costs.

Control costs

☐ **Terminate and launch**

Terminate and launch instances at the same time. This allows you to go below your desired capacity by a given percentage and may temporarily reduce availability.

Flexible

☐ **Custom behavior**

Set custom values for the minimum and maximum amount of available capacity. This gives you greater flexibility in setting how far below and over your desired capacity EC2 Auto Scaling goes when replacing instances.

4. Click **Next**

no policy

For rebalancing events, new instances will launch before terminating others. For all other events, instances terminate and launch at the same time.

Launch before terminating

Launch new instances and wait for them to be ready before terminating others. This allows you to go above your desired capacity by a given percentage and may temporarily increase costs.

Terminate and launch

Terminate and launch instances at the same time. This allows you to go below your desired capacity by a given percentage and may temporarily reduce availability.

Custom behavior

Set custom values for the minimum and maximum amount of available capacity. This gives you greater flexibility in setting how far below and over your desired capacity EC2 Auto Scaling goes when replacing instances.

Additional capacity settings

Capacity Reservation preference [info](#)

Select whether you want Auto Scaling to launch instances into an existing Capacity Reservation or Capacity Reservation resource group.

☒ **Default**

Auto Scaling uses the Capacity Reservation preference from your launch template.

☐ **None**

Instances will not be launched into a Capacity Reservation.

☐ **Capacity Reservations only**

Instances will only be launched into a Capacity Reservation. If capacity isn't available, the instances fail to launch.

☐ **Capacity Reservations first**

Instances will attempt to launch into a Capacity Reservation first. If capacity isn't available, instances will run in On-Demand capacity.

Additional settings

Instance scale-in protection

If protect from scale-in is enabled, newly launched instances will be protected from scale in by default.

☐ **Enable instance scale-in protection.**

Monitoring [info](#)

☐ **Enable group metrics collection within CloudWatch**

Default instance warmup [info](#)

The amount of time that CloudWatch metrics for new instances do not contribute to the group's aggregated instance metrics, so their usage data is not reliable yet.

☐ **Enable default instance warmup**

Auto Scaling group deletion protection - new [info](#)

Configure how this Auto Scaling group can be deleted.

☒ **None (default)**

The Auto Scaling group can be deleted.

☐ **Prevent force deletion**

The Auto Scaling group cannot be force deleted. To delete the group, all instances must be detached or terminated, and all warm pools must be deleted.

☐ **Prevent all deletion**

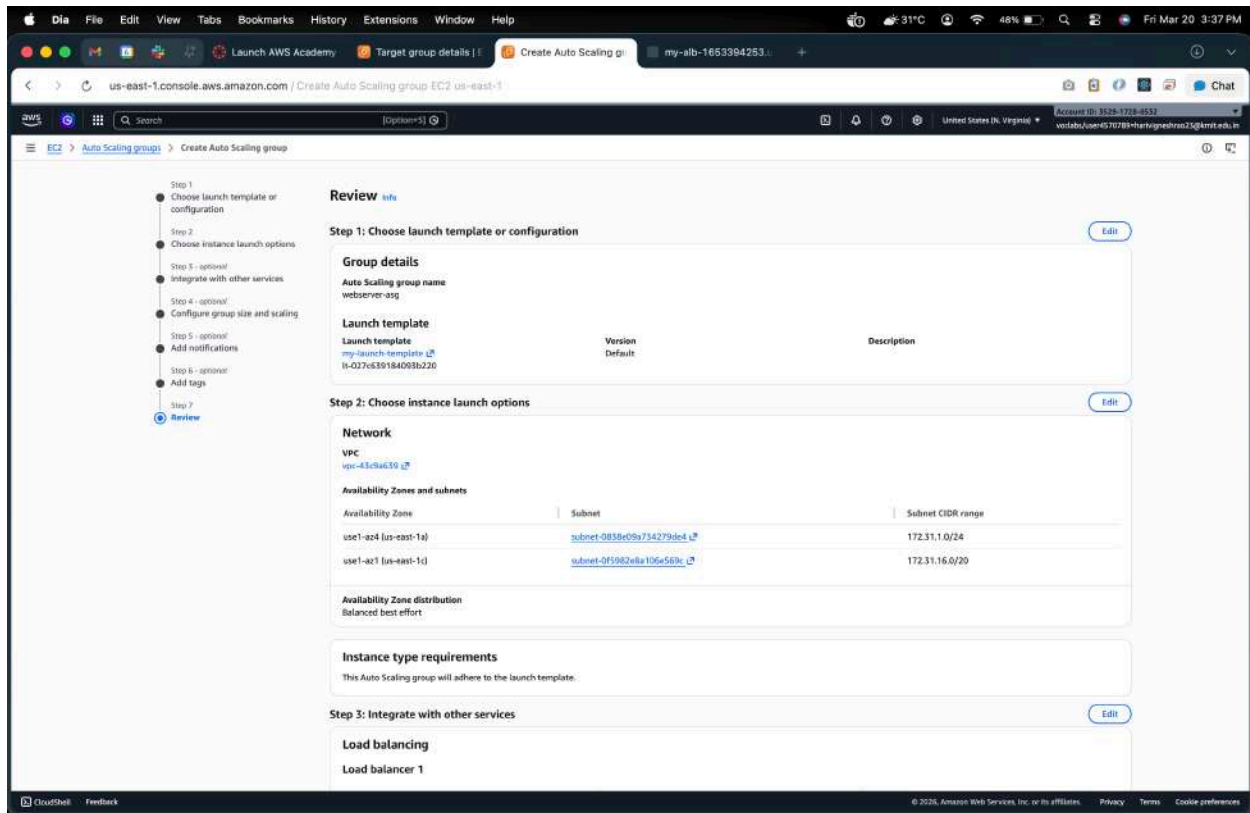
The Auto Scaling group cannot be deleted.

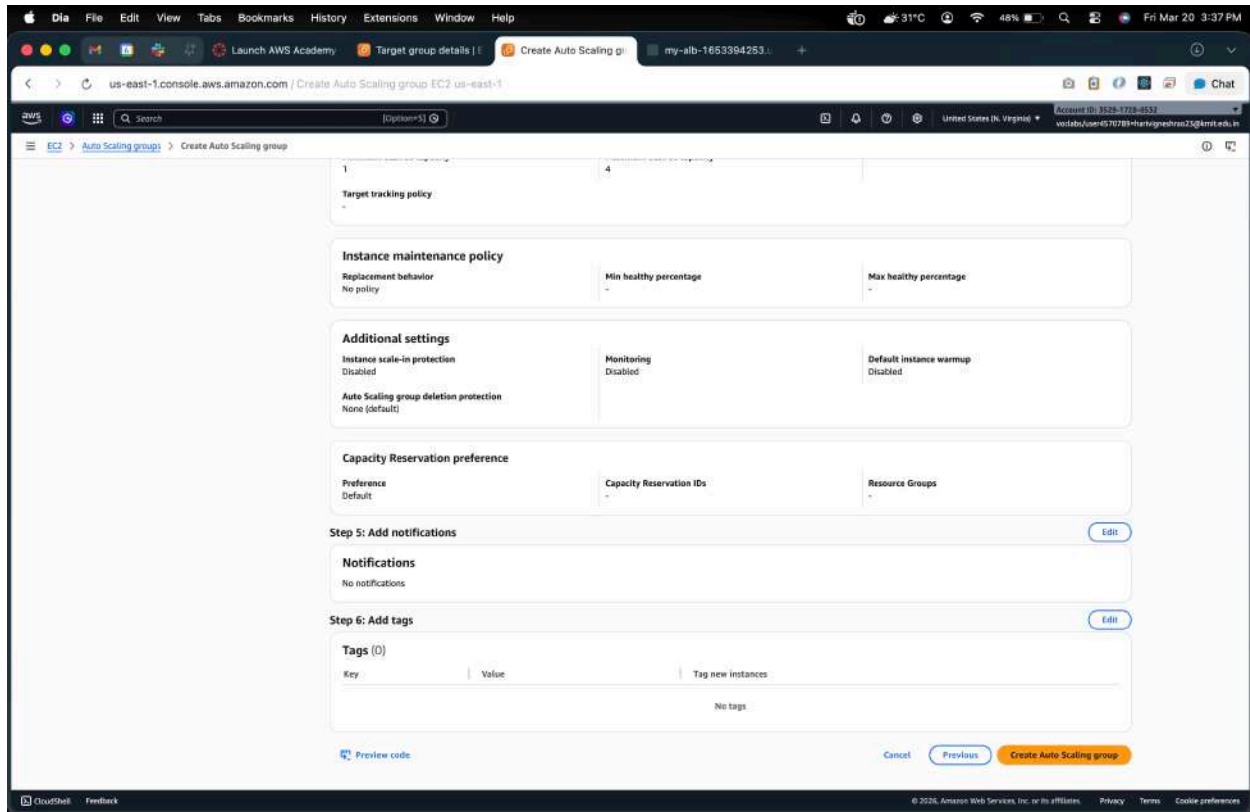
Cancel [Skip to review](#) [Previous](#) **Next**

Step 3.3: Review and Create ASG

1. Review all settings
2. Click **Create Auto Scaling Group**

Auto Scaling Group is now ready and attached to your Load Balancer.





Step 4: Verify Load Balancer and Auto Scaling

1. Go to **Load Balancers** → **Select your Load Balancer**
2. Click **Instances** → **Target Group** → **Registered Targets**
 - You will see **2 healthy instances** initially

The screenshot displays the AWS Management Console interface for a Target Group named 'web-servers-tg'. The left sidebar shows the navigation menu with categories like Instances, Images, Elastic Block Store, Network & Security, Load Balancing, and Auto Scaling. The main content area shows the 'web-servers-tg' details, including its target type (Instance), IP address type (IPv4), and protocol (HTTP: 80). It also shows the distribution of targets by Availability Zone (AZ) and a table of registered targets.

Target Group Details:

- Target type:** Instance
- IP address type:** IPv4
- Protocol:** HTTP: 80
- Protocol version:** HTTP1
- VPC:** vpc-43c9a518
- Load balancer:** None associated

Targets Summary:

- Total targets:** 2
- Healthy:** 2
- Unhealthy:** 0
- Unused:** 0
- Initial:** 0
- Draining:** 0

Distribution of targets by Availability Zone (AZ):

Select values in this table to see corresponding filters applied to the Registered targets table below.

Registered targets (2):

Instance ID	Name	Port	Zone	Health status	Health status details	Admin...	Overrid...	Launch...	Anomaly detectio...
i-b0c260ed5ea755ddde	webserver-2	80	us-east-1a (use...)	Healthy		-	-	March 20, ...	Normal
i-b0c2c592fab6a3005	webserver-1	80	us-east-1a (use...)	Healthy		-	-	March 20, ...	Normal

3. Check **Auto Scaling Group** → **Instances**

- You may see up to **4 instances** if scaling triggers

The screenshot shows the AWS Management Console for the 'us-east-1' region. The left sidebar contains navigation links for various AWS services. The main content area displays the details for the 'web-servers-tg' target group. The 'Details' section shows the target type as 'Instance', IP address type as 'IPv4', and protocol as 'HTTP: 80'. It also indicates that there are 4 total targets, all of which are 'Healthy'. Below this, a 'Distribution of targets by Availability Zone (AZ)' section shows that the targets are distributed across three AZs: us-east-1c, us-east-1a, and us-east-1b. The 'Targets' section lists four registered targets, each with its instance ID, name, port, zone, health status, and administrative details.

Instance ID	Name	Port	Zone	Health status	Health status details	Administrative details	Override details	Launch time	Anomaly detection
i-06a06c03c2f41b40	-	80	us-east-1c (us-east-1)	Healthy	-	No override	No override is c...	March 20, 2026...	Normal
i-0942967a7e7b0d296	-	80	us-east-1a (us-east-1)	Healthy	-	No override	No override is c...	March 20, 2026...	Normal
i-bb-26b0d5e4755ddde	webserver-2	80	us-east-1a (us-east-1)	Healthy	-	No override	No override is c...	March 20, 2026...	Normal
i-00c7c892f6b2a3005	webserver-1	80	us-east-1a (us-east-1)	Healthy	-	No override	No override is c...	March 20, 2026...	Normal

The screenshot shows the AWS Management Console for the 'us-east-1' region. The left sidebar contains navigation links for various AWS services. The main content area displays the details for the 'webserver-asg' auto scaling group. The 'Capacity overview' section shows the desired capacity as 2, scaling limits from 1 to 4, and the desired capacity type as 'Units (number of instances)'. The 'Instances' section lists two instances, both of which are 'Healthy'. Below this, the 'Instance lifecycle policy for lifecycle hooks' section shows that there are no lifecycle hooks currently configured.

Instance ID	Lifecycle	Instance type	Weighted capacity	Launch template	Availability Zone	Health status	Protected from
i-06a06c03c2f41b40	InService	t3.micro	-	my-launch-template	us-east-1c (us-east-1)	Healthy	-
i-0942967a7e7b0d296	InService	t3.micro	-	my-launch-template	us-east-1a (us-east-1)	Healthy	-

4. Test auto recovery:

- Terminate one of the instances

The screenshot shows the AWS Management Console for the 'us-east-1' region. The 'Instances' page is active, displaying a table of EC2 instances. A dropdown menu is open for the instance 'webserver-1' (ID: i-00c2c692fabba3005), showing options: Stop Instance, Start Instance, Reboot Instance, Terminate Instance, and Terminate (delete) instance. The instance details for 'webserver-1' are visible below the table.

Name	Instance ID	Instance state	Instance type	Status check	Alarm status	Availability Zone	Public IPv4
webserver-1	i-00c2c692fabba3005	Running	t3.micro	3/3 checks passed	View alarms +	us-east-1a	ec2-54-91-2
webserver-2	i-04c268ed5ea7554de	Running	t3.micro	3/3 checks passed	View alarms +	us-east-1a	ec2-34-203
	i-06a06ce9312f41b40	Running	t3.micro	3/3 checks passed	View alarms +	us-east-1c	-
	i-09d2967ae70bed296	Running	t3.micro	3/3 checks passed	View alarms +	us-east-1a	-

Instance summary for i-00c2c692fabba3005 (webserver-1)

- Instance ID:** i-00c2c692fabba3005
- Instance state:** Running
- Instance type:** t3.micro
- Public IPv4 address:** 54.91.222.59 | [open address](#)
- Private IP DNS name (IPv4 only):** ip-172-31-1-87.ec2.internal
- Public DNS:** ec2-54-91-222-59.compute-1.amazonaws.com | [open address](#)
- Hostname type:** IP name: ip-172-31-1-87.ec2.internal
- Answer private resource DNS name:** -
- Auto-assigned IP address:** -
- Private IPv4 addresses:** 172.31.1.87
- Elastic IP addresses:** -

- After a few seconds, **ASG** will launch a new instance automatically
- Traffic will automatically be **distributed by the Load Balancer**

Instances (5) info

Name	Instance ID	Instance state	Instance type	Status check	Alarm status	Availability Zone	Public IPv4 DNS	Public IPv4 ...	Elastic IP	IPv6 IPs
webserver-1	i-06c2c652fbb3a3005	Terminated	t3.micro	-	View alarms +	us-east-1a	-	-	-	-
webserver-2	i-04c268e05a755d4e	Terminated	t3.micro	-	View alarms +	us-east-1a	ec2-34-203-200-66.co...	34.203.200.66	-	-
	i-06a05ce93c2f41b40	Running	t3.micro	3/3 checks passed	View alarms +	us-east-1c	-	-	-	-
	i-02c2e4f35829704d1	Running	t3.micro	Initializing	View alarms +	us-east-1c	-	-	-	-
	i-09d2967ac76bed296	Running	t3.micro	3/3 checks passed	View alarms +	us-east-1a	-	-	-	-

Select an instance

webserver-asp Capacity overview

arn:aws:autoscaling:us-east-1:35291728053:autoScalingGroup:0aeb049f-cb0c-4406-9e41-2a75ed67f004:autoScalingGroupName/webserver-asp

Desired capacity: 2 | Scaling limits: 1 - 4 | Desired capacity type: Units (number of instances) | Status: -

Date created: Fri Mar 20 2026 15:37:55 GMT+0530 (India Standard Time)

Details | Integrations | Automatic scaling | **Instance management** | Instance refresh | Activity | Monitoring | Tags

Instances (3)

Instance ID	Lifecycle	Instance type	Weighted capacity	Launch template...	Availability Zone	Health status	Protected from
i-02c2e4f35829704d1	InService	t3.micro	-	my-launch-template_1	us-east-1c (us-east-1c)	Healthy	-
i-06a05ce93c2f41b40	Terminating	t3.micro	-	my-launch-template_1	us-east-1c (us-east-1c)	Unhealthy	-
i-09d2967ac76bed296	InService	t3.micro	-	my-launch-template_1	us-east-1a (us-east-1a)	Healthy	-

Instance lifecycle policy for lifecycle hooks

Controls instance behavior when an instance transitions through its lifecycle states.

Termination hook abandon behavior: Terminate (default)

Lifecycle hooks (0)

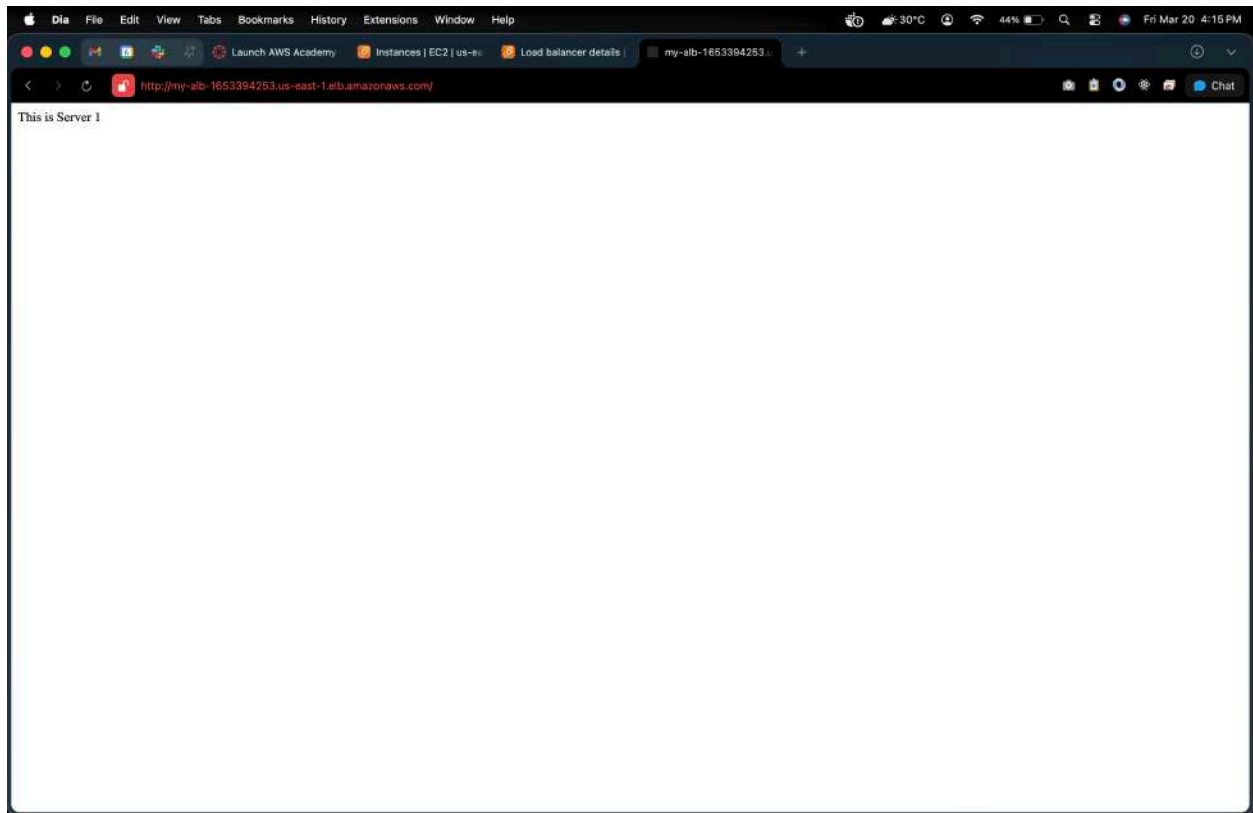
No lifecycle hooks are currently configured.

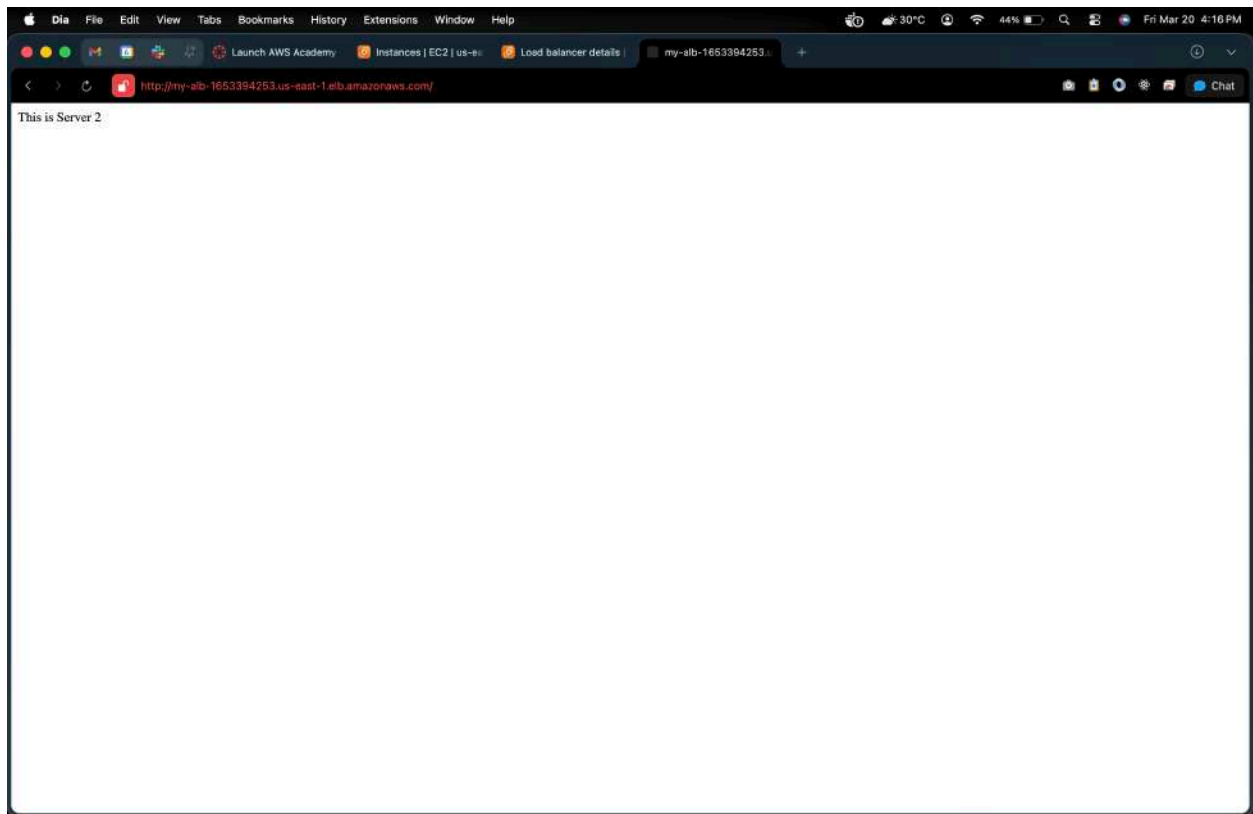
Lifecycle hooks help you perform custom actions on instances as they launch and before they terminate.

Step 5: Test the Load Balancer

1. Copy the **Load Balancer DNS Name** from ALB
2. Open in a browser → You should see your web page from one of the instances
3. Refresh multiple times → traffic alternates between instances

This confirms **Auto Scaling + Load Balancer** is working correctly.





Summary of Flow

1. **Create AMI** → snapshot of working EC2
2. **Create Launch Template** → using AMI, security group
3. **Create Auto Scaling Group** → attach Launch Template, set min/desired/max, attach Load Balancer
4. **Verify instances** → ASG launches/replaces instances automatically
5. **Test Load Balancer** → distributes traffic between all active instances